24CSEN2031 – Database Management Systems

Unit-1

5/10/2026

Dr. Figlu Mohanty
Assistant Professor
Dept. of CSE
GITAM, Hyderabad

Important Terminology

- **Data:** Any raw fact or figures.
  - **Data can be text, numbers, image, audio, video, statistics and so on.**
- **Database:** Collection of related data.
- **Management System**: Set of programs to store and retrieve these data.
- **Database Management System (DBMS**): A software package/ system to facilitate the creation and maintenance of a computerized database.

Cont…

Database System is a

- Computerized record-keeping system
- Supports operations
  - Add or delete files to the database
  - Insert, retrieve, remove, or change data in database

For example: MySQL, Oracle, etc are a very popular commercial database which is used in different applications.

Data vs. Information

Parameter	Data	Information
Description	Variables, either quantitative or qualitative, that aid in the development of conclusions or ideas.	It's a collection of facts that has meaning and news.
Dependence	It doesn't depend on information.	It's dependent on data.
Contains	Unprocessed raw factors.	It's processed in a meaningful manner.
Example	**Each student's test score is one piece of data.**	**The average score of a class or of the entire school is information that can be derived from the given data.**

Difference between File System and DBMS

- **File System Approach:**
  - File based systems were an early attempt to computerize the manual system.
  - A File Processing System is a method or a tool which facilitates storing, accessing and modifying data from numerous files in a computer system.
  - All data is stored in the form of files. All files are categorized and sorted accordingly. The file names are closely related to one another and are organized in such a way that they are easily accessible.
  - **Example: Suppose a university stores student records in one file (students.txt) and course enrollments in another (enrollments.txt). If a student changes their name, this must be updated manually in every file that references the student, which may cause inconsistencies.**

Limitation of File System

Limitation	Description
Data Redundancy	Same data may be stored in multiple files, wasting space.
Data Inconsistency	Changes in one file might not reflect in others.
Lack of Integration	Difficult to combine data from multiple sources.
Poor Data Security	No centralized access control.
Difficult to Query	No query language like SQL; complex querying must be coded manually.
Concurrency Issues	Managing simultaneous access is complex and error-prone.

- **DBMS:**
  - A database approach is a well-organized collection of data that are related in a meaningful way which can be accessed by different users but stored only once in a system.
  - The various operations performed by the DBMS system are: Insertion, deletion, selection, sorting etc.

Basis	File System	DBMS
Structure	The file system is a way of arranging the files in a storage medium within a computer.	DBMS is software for managing the whole database.
Data Redundancy	Redundant data can be present in a file system.	In DBMS there is no redundant data.
Backup and Recovery	It doesn't provide backup and recovery of data if it is lost.	It provides backup and recovery of data even if it is lost.
Query processing	There is no efficient query processing in the file system.	Efficient query processing is there in DBMS.
Consistency	There is less data consistency in the file system.	There is more data consistency because of the process of normalization.
Complexity	It is less complex as compared to DBMS.	It has more complexity in handling as compared to the file system.
Security Constraints	File systems provide less security in comparison to DBMS.	DBMS has more security mechanisms as compared to file systems.
Cost	It is less expensive than DBMS.	It has a comparatively higher cost than a file system.

Advantages of DBMS

- Remove data redundancy
- Improved data security
- Fast Data Sharing
- Minimize data inconsistency
- Improved data accessing

Disadvantages of DBMS

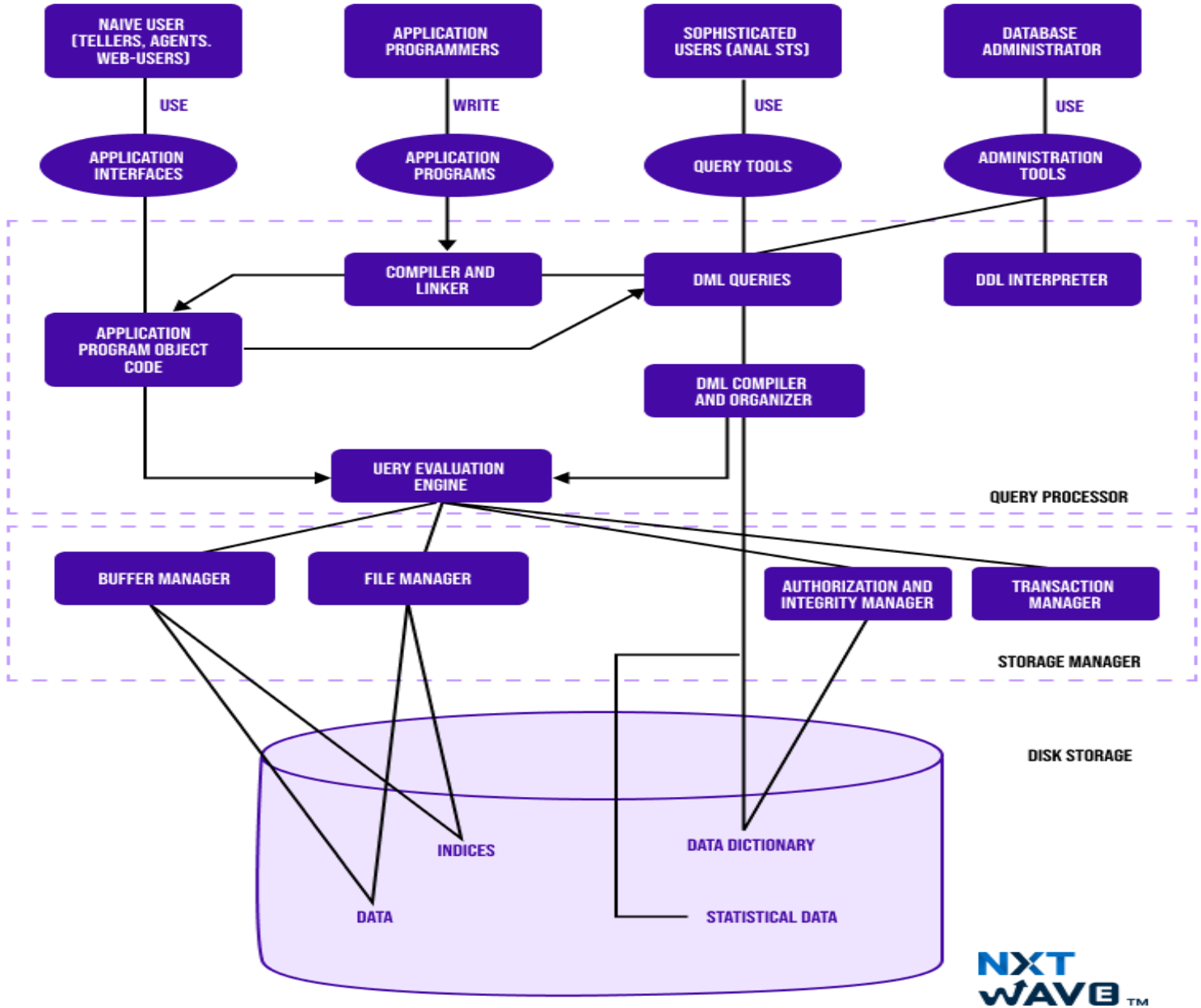
- More overhead cost
- Management complexity
- Maintaining concurrency

Cont…

- **New functionality is being added to DBMSs in the following areas:**
  - Scientific Applications
  - Image Storage and Management
  - Audio and Video data management
  - Data Mining
  - Time Series and Historical Data Management

DBMS structure

- The DBMS accepts SQL commands generated from a variety of user interfaces, produces query evaluation plans, executes these plans against the database, and returns the answers.
- When a user issues a query, the parsed query is presented to a query optimizer, which uses information about how the data is stored to produce an efficient execution plan for evaluating the query.
- An execution plan is a blueprint for evaluating a query, usually represented as a tree of relational operators.


NAIVE USER (TELLERS, AGENTS. WEB-USERS)
APPLICATION PROGRAMMERS
SOPHISTICATED USERS (ANAL STS)
DATABASE ADMINISTRATOR
USE
WRITE
USE
USE
APPLICATION INTERFACES
APPLICATION PROGRAMS
QUERY TOOLS
ADMINISTRATION TOOLS
COMPILER AND LINKER
DML QUERIES
DDL INTERPRETER
APPLICATION PROGRAM OBJECT CODE
DML COMPILER AND ORGANIZER
UERY EVALUATION ENGINE
QUERY PROCESSOR
BUFFER MANAGER
FILE MANAGER
AUTHORIZATION AND INTEGRITY MANAGER
TRANSACTION MANAGER
STORAGE MANAGER
DISK STORAGE
INDICES
DATA DICTIONARY
DATA
STATISTICAL DATA

Database Administrator

- DBA can be a single person or it can be a group of person.
- **Tasks of DBA:**
  –Creating the schema
  –Changing the schema
  –Specifying integrity constraints
- –Storage structure and access method definition
- –Granting permission to other users.
- –Monitoring performance
- –Routine Maintenance

Naïve/Parametric End user

- Parametric End Users are the unsophisticated who don't have any DBMS knowledge but they frequently use the data base applications in their daily life to get the desired results.
- For examples, Railway's ticket booking users are naive users. Clerks in any bank is a naive user because they don't have any DBMS knowledge but they still use the database and perform their given task.

Sophisticated End users

- Sophisticated users can be engineers, scientists, business analyst, who are familiar with the database.
- They can develop their own data base applications according to their requirement.
- They don’t write the program code but they interact the data base by writing SQL queries directly through the query processor.

Application Programmer

- Application Program are the back end programmers who writes the code for the application programs.
- They are the computer professionals.
- These programs could be written in Programming languages such as Visual Basic, Developer, C, FORTRAN, COBOL etc.

1. Disk Storage:

- All data or information stored in disk storage.
- Various components of disk storage
  - Data: Data files which stores the database itself.
  - Data dictionary: It contains meta data. data about data and Schema of table is an example of meta
  - Indices: It provides fast access to data items that holds particular values.
  - Statistical data: Stores information/statistics of data stored.

2. Storage manager

a) Authorization and integrity manager
- Checks authority of users accessing the data.
- It tests integrity and constraints of data

b) Transaction manager
- It ensures that the database remains in a consistent state despite of failure.
- Concurrent execution in maintained without any conflict.

c) File manager
- It manages allocation of space on disk storage.
- Information stored on disk is represented using database.

d) Buffer manager
- It is responsible for fetching data from disk storage into main memory.
- It decides what data to cache the main memory

3. Query processor

- Query processor is an important part of the database system
- It helps the database system to simplify and facilitate access to data.

a) DDL Interpreter: It interprets DDL statements into low level.
b) DML Compiler: It translates DML query statement in query language into low-level instruction
c) Query evaluation engine: Executes low level instruction generated by DML compiler.

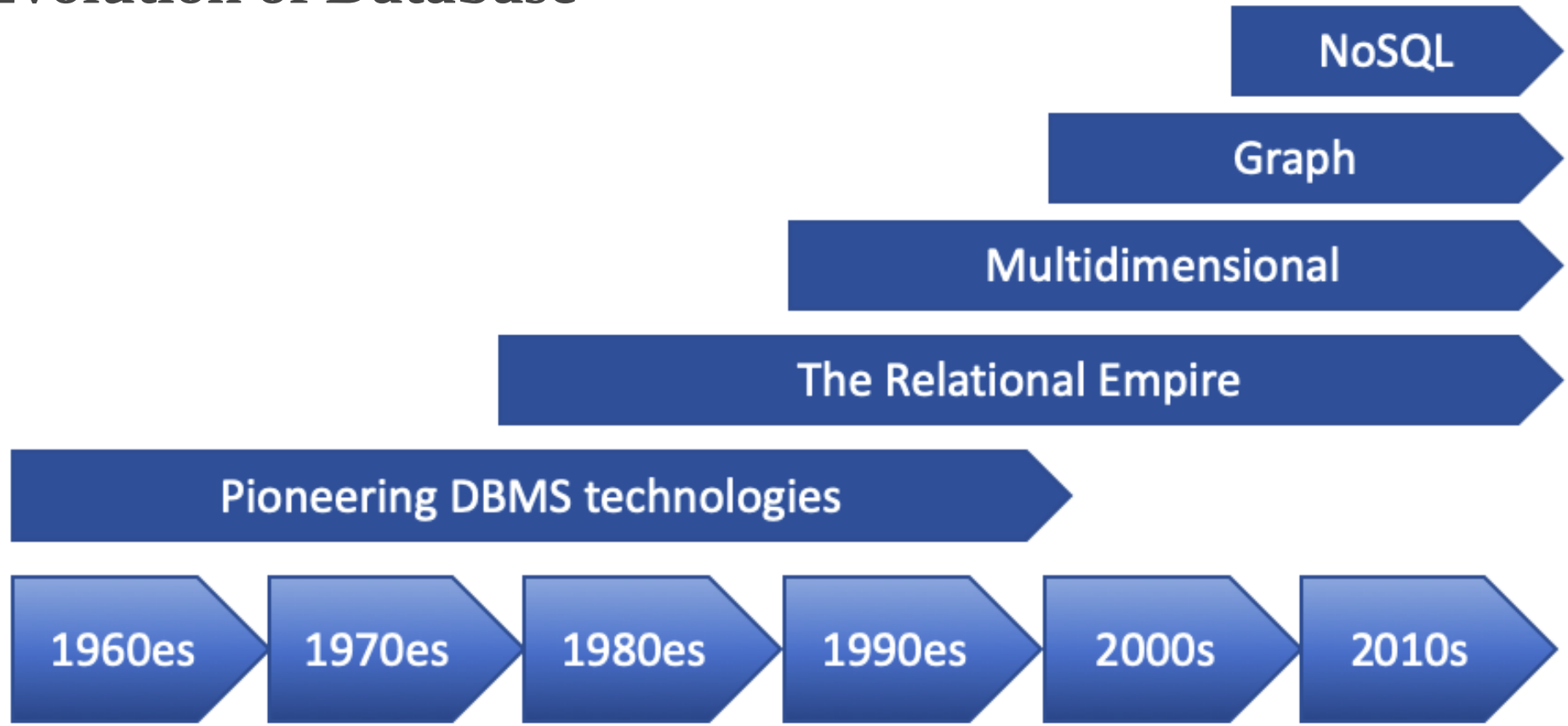
Data Models

- A collection of concepts that can be used to describe the structure of a database.
- Most data models also include a set of basic operations for specifying retrievals and updates on the database.
- Data model defines the logical representation of a database.

Evolution of Data Model

- Representational or implementation data models are the models used most frequently in traditional commercial DBMSs.
- These include:
  - Hierarchical data model
  - Network data model
  - Relational data model
  - Entity-Relational
  - Object-based data model
  - Key-value data model
  - XML data model

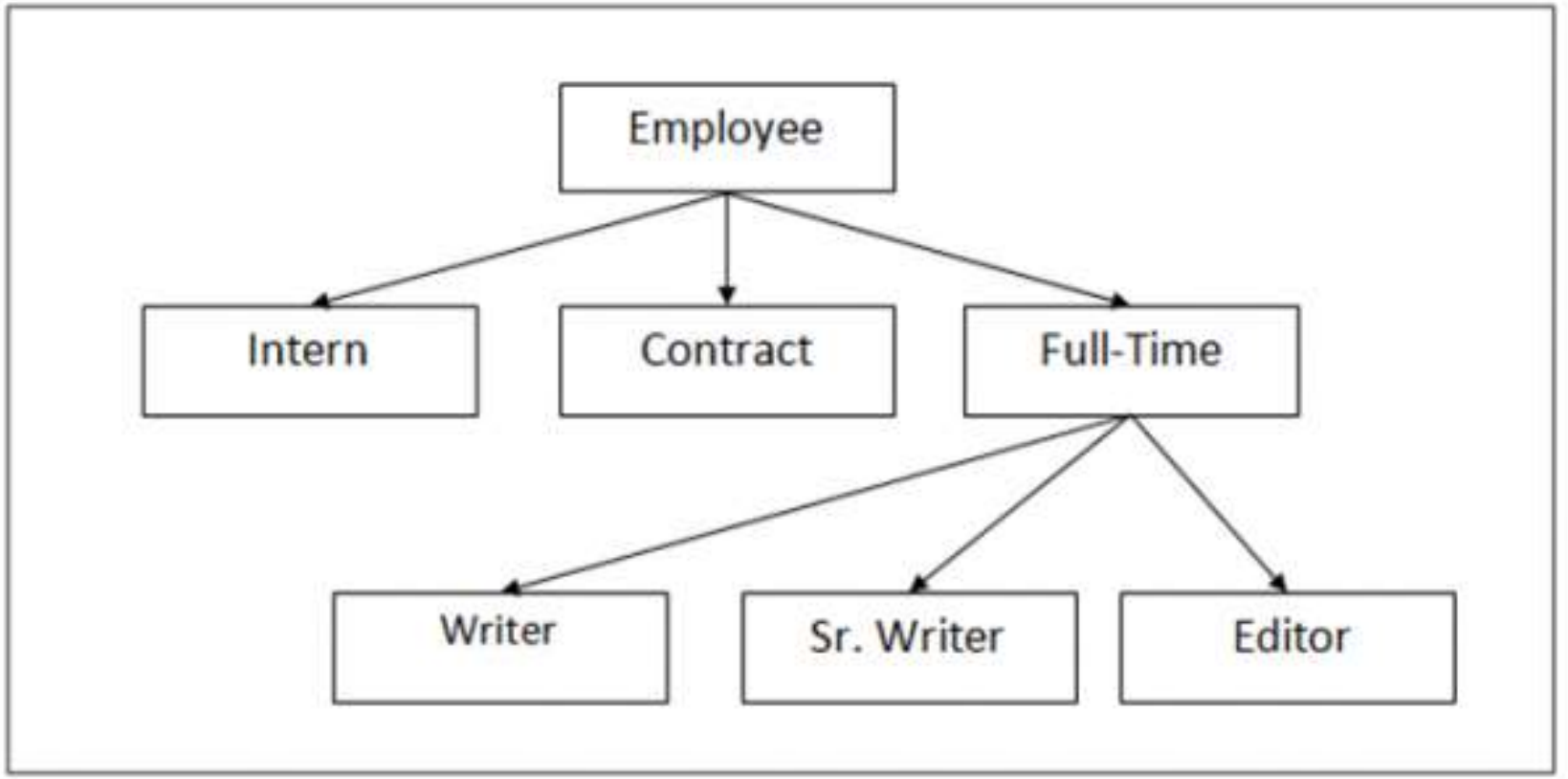
Evolution of Database



Hierarchical data model

- One of the first and most popular Hierarchical Model is Information Management System (IMS), developed by IBM.
- The relationship can be defined in the form of parent child type.
- The design of the hierarchical model is simple.
- The hierarchy begins at the root, which contains root data, and then grows into a tree as child nodes are added to the parent node.
- A parent node exists for each child node. However, a parent node might have several child nodes. It is not permitted to have more than one parent.
- Even for large volumes of data, this model works perfectly.

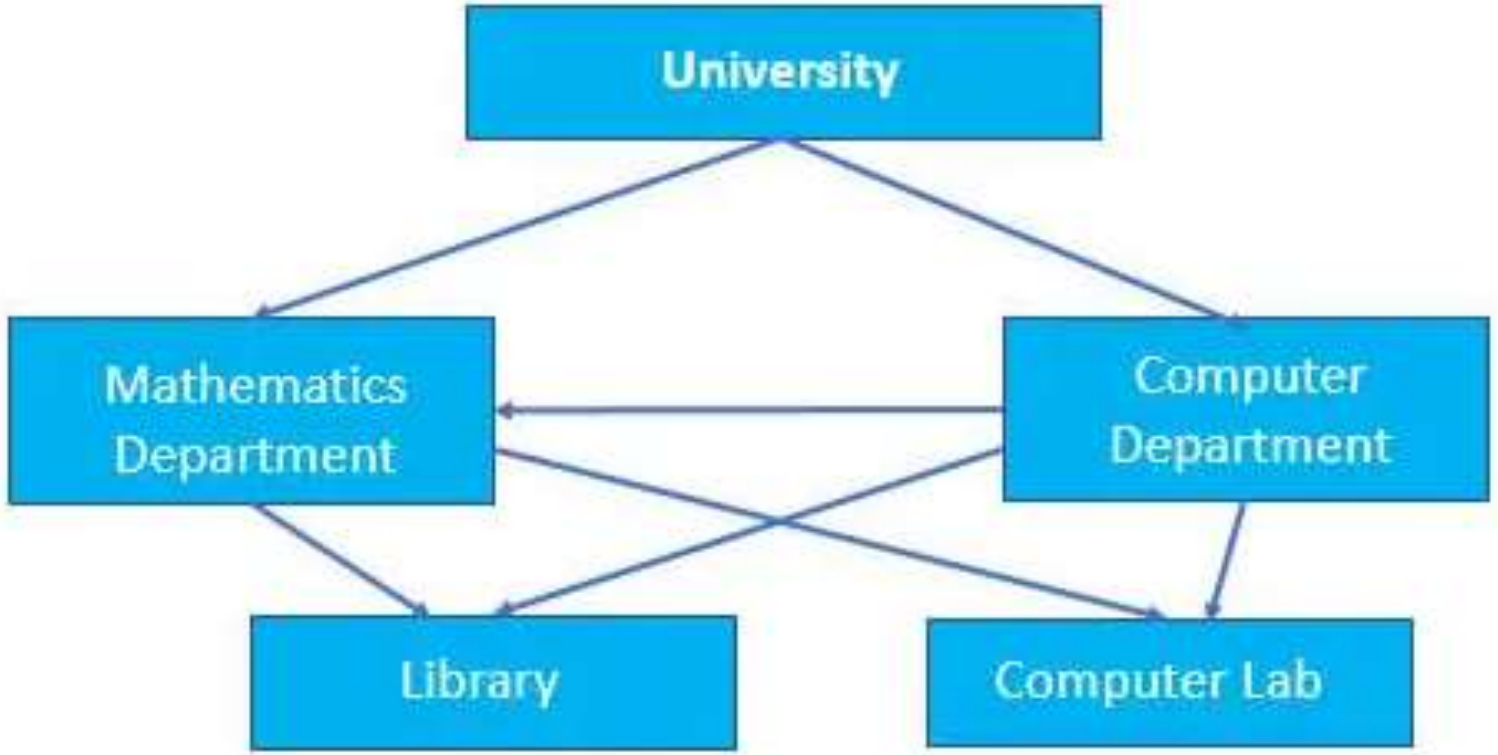
Hierarchical data model (cont…)



Network data model

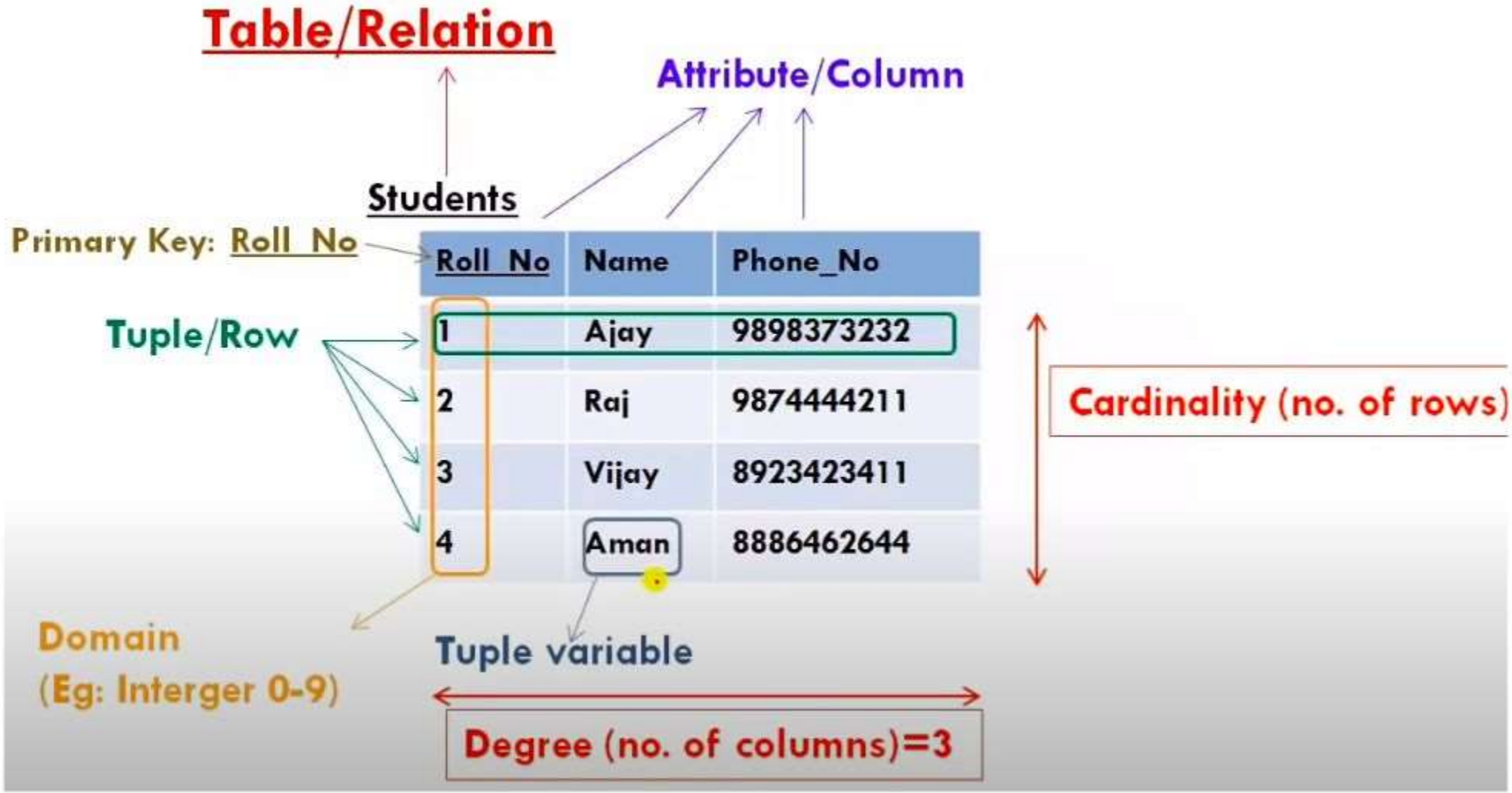
- The Network Model has graph and links.
- The relationship can be defined in the form of links and it handles many-to-many relations. This itself states that a record can have more than one parent.
- The model can handle one-one, one-to-many, many-to-many relationships.
- Pointers bring complexity since the records are based on pointers and graphs.
- In comparison to the hierarchical model, data can be retrieved faster. This is because there may be more than one path to a given node. As a result, the data can be accessed in a variety of ways.

Network data model (cont…)



Relational data model

- A relational model groups data into one or more tables. These tables are related to each other using common records.
- The data is represented in the form of rows and columns i.e. tables.
- Changes in the database do not require you to affect the complete database.
- Implementation of a Relational Model is easy.
- Popular Relational Database Management Systems:
  - IBM – DB2 and Informix Dynamic Server
  - Oracle – Oracle and RDB
  - Microsoft – SQL Server and Access


Table/Relation
Attribute/Column
Students
Primary Key: Roll_No
Roll_No
Name
Phone_No
Tuple/Row
1
Ajay
9898373232
2
Raj
9874444211
3
Vijay
8923423411
4
Aman
8886462644
Cardinality (no. of rows)
Domain
(Eg: Interger 0-9)
Tuple variable
Degree (no. of columns)=3

Relational data model (cont…)

EMPLOYEE

ID_NO	NAME	ADDRESS	ROLL_NO	AGE
C1	RIYA	DELHI	15	20
C2	SUNITA	GURGAON	16	22
C3	ASHWANI	ROHTAK	12	18
C4	PREETI	DELHI		25

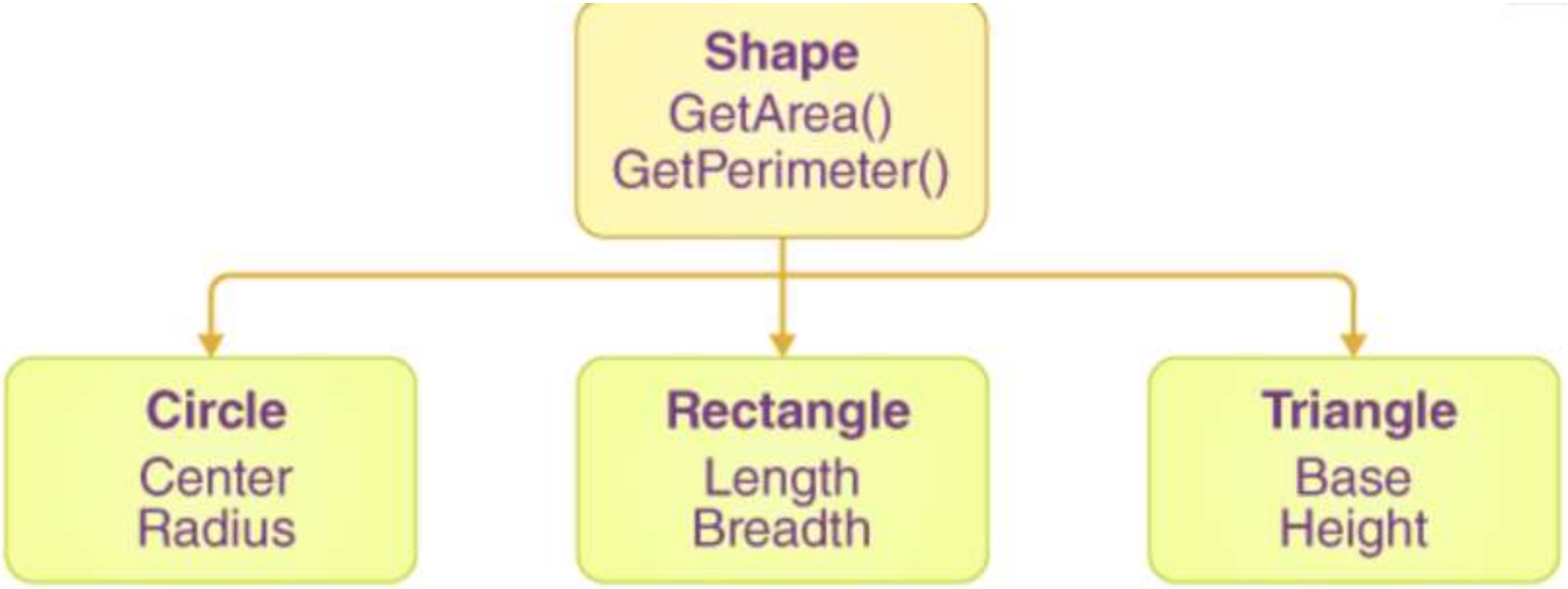
Facts of Relation:

- **Attribute**: It refers to every column present in a table. The attributes refer to the properties that help us define a relation. E.g., Employee_ID, Student_Rollno, SECTION, NAME, etc.
- **Tables** – In the case of the relational model, all relations are saved in the table format, and it is stored along with the entities. A table consists of two properties: columns and rows. While rows represent records, the columns represent attributes.
- **Tuple** – It is a single row of a table that consists of a single record. The relation above consists of four tuples.
- **Degree**: It refers to the total number of attributes/columns that are there in the relation. The EMPLOYEE relation defined here has degree 5.
- **Cardinality**: It refers to the total number of rows present in the given table. The EMPLOYEE relation defined here has cardinality 4.

Object-based data model (OODM)

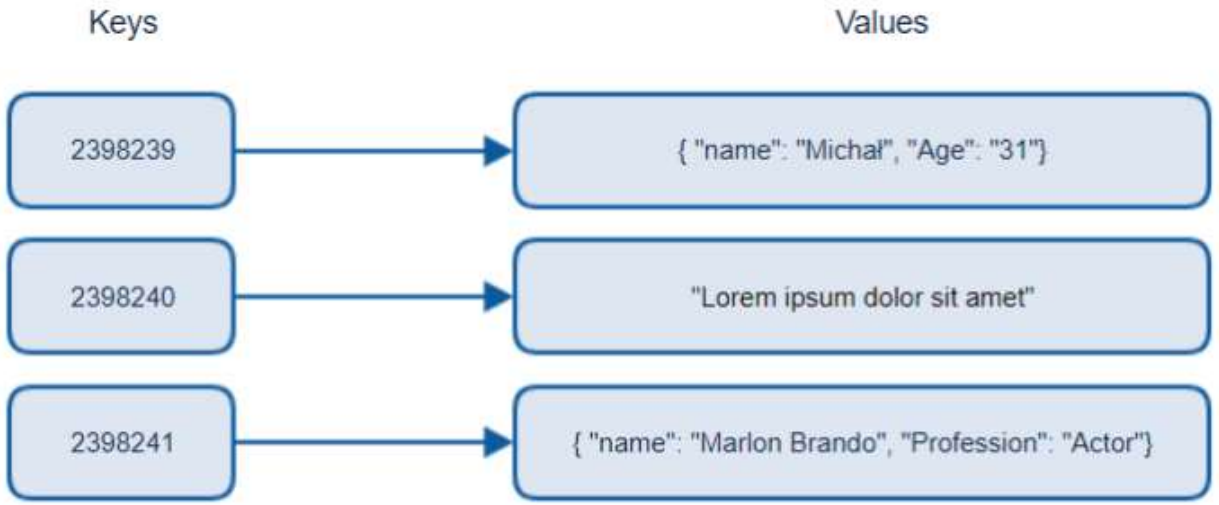
- In Object Oriented Data Model, data and their relationships are contained in a single structure which is referred as object in this data model.
- In this, real world problems are represented as objects with different attributes.
- **Object Oriented Data Model = Combination of Object Oriented Programming + Relational database model**

OODM (cont…)


Shape
GetArea()
GetPerimeter()
Circle
Center
Radius
Rectangle
Length
Breadth
Triangle
Base
Height

Key-value data model

- A key-value database is a type of non relational database that uses a simple key-value method to store data.
- A key-value database stores data as a collection of key-value pairs in which a key serves as a unique identifier.
- Both keys and values can be anything, ranging from simple objects to complex compound objects.
- A type of NoSQL DBMS that store data as a mapping of keys to values and are optimized for high-speed data retrieval.
- Example: **Session Management in Web Applications Store user session data: SessionID → Data**



XML Data model

- XML database is a data persistence software system used for storing the huge amount of information in XML format. It provides a secure place to store XML documents.
- You can query your stored data by using XQuery, export and serialize into desired format. XML databases are usually associated with document-oriented databases.
- It is particularly useful as a data format when an application

must communicate with another application,

or integrate information from several other applications.

Example: Web Services, Configuration Files..

```
<student>
  <student_id>S101</student_id>
  <name>John Smith</name>
  <email>john@example.com</email>
  <courses>
    <course>
      <course_id>CSE101</course_id>
      <course_name>Data Structures</course_name>
    </course>
    <course>
      <course_id>CSE102</course_id>
      <course_name>Database Systems</course_name>
    </course>
  </courses>
</student>
```

DBMS vs. RDBMS

DBMS

- DBMS applications store **data as file**.
- **Normalization is not** present in DBMS.
- In DBMS, data is generally stored in either a hierarchical form or a navigational form.
- **no relation between the tables**.

RDBMS

- RDBMS applications store **data in a tabular form**.
- **Normalization is** present in RDBMS.
- In RDBMS, the tables have an identifier called primary key and the data values are stored in the form of tables.
- A **relationship** between these data values will be stored in the form of a table as well.

Schema & Instance

- The structure of a database is called the **database schema**, which is specified during database design and is not expected to change frequently.
- **Database Instance:** The actual data stored in a database at a particular moment of time. Also called database state (or occurrence).

Note:

- **Schema** = Database **structure**
- **Instance** = Database **content**

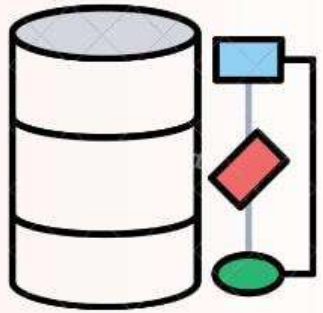
DIFFERENCE BETWEEN SCHEMA AND INSTANCE

SCHEMA

NAME	ROLL NUMBER

INSTANCE

NAME	ROLL NUMBER
NAME-1	12
NAME-2	13
NAME-3	17



CODING NINJAS

Aspect	Schema	Instance
Definition	The **structure/design** of the database	The **actual data** stored in the database at a given moment
Type	**Intensional** (defines what it should be)	**Extensional** (represents current data)
Change	Rarely changes (only if design changes)	Changes frequently (as data is inserted, deleted, or updated)
Example	Table definitions, field names, data types	Rows/records stored in those tables

Three-Schema Architecture/ Levels of Abstraction

1. **Internal Schema (Physical Level)**

What it is: Describes how data is actually stored on disk (files, indexes, data blocks).

Who uses it: Database administrators (DBAs)

Focus: Storage format, access methods, compression, indexing.

Example: The fact that a Student table is stored as a B-tree indexed file, split into blocks on a hard disk.

cont…

2. Conceptual Schema (Logical Level)

What it is: Describes what data is stored and the relationships among the data.
Who uses it: Database designers and programmers
Focus: Tables, attributes, data types, constraints (PK, FK), relationships.

Example: A Student table has fields: Roll_No, Name, Email

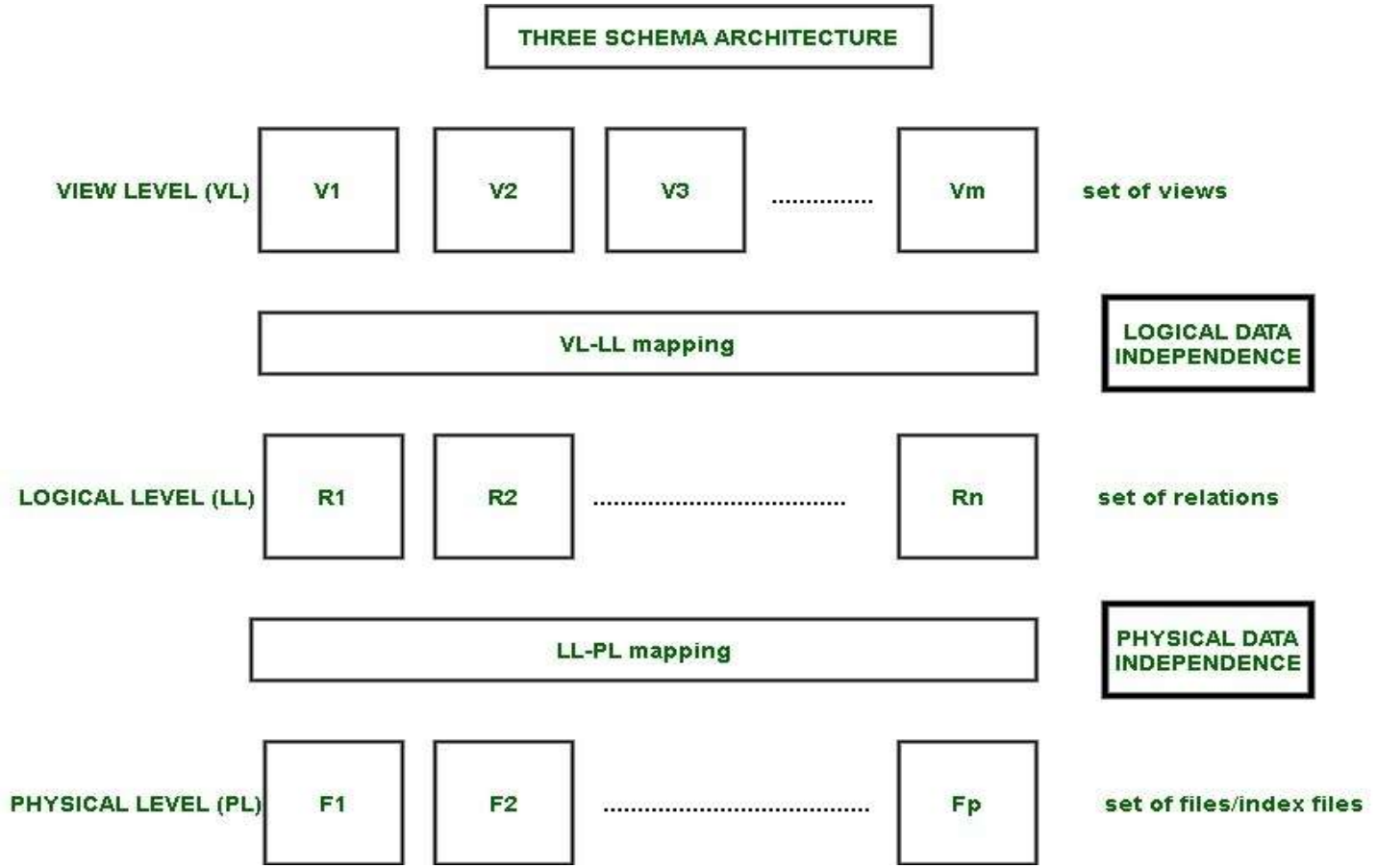
cont…

3. External Schema (View Level)

What it is: Describes part of the database relevant to a particular user or application.
Who uses it: End users
Focus: Custom user views, hiding irrelevant data and ensuring security.

Example: A student can see only the faculty's name, contact or email but not the salary or DOJ etc.

Cont…


THREE SCHEMA ARCHITECTURE
VIEW LEVEL (VL)
V1
V2
V3
.....
Vm
set of views
VL-LL mapping
LOGICAL DATA INDEPENDENCE
LOGICAL LEVEL (LL)
R1
R2
.....
Rn
set of relations
LL-PL mapping
PHYSICAL DATA INDEPENDENCE
PHYSICAL LEVEL (PL)
F1
F2
.....
Fp
set of files/index files

Data Independence

- Data Independence is defined as a property of DBMS that helps you to change the Database schema at one level of a database system without requiring to change the schema at the next higher level.
- Data independence helps you to keep data separated from all programs that make use of it.

Types of Data Independence

- **Logical Data Independence:** The capacity to change the conceptual schema without having to change the external schemas and their application programs.
- Due to Logical independence, any of the below change will not affect the external layer.
  - Add/Modify/Delete a new attribute, entity or relationship is possible without a rewrite of existing application programs
  - Merging two records into one
  - Breaking an existing record into two or more records

Cont…

- **Physical Data Independence:** The capacity to change the physical schema without having to change the conceptual schema.
- Due to Physical independence, any of the below change will not affect the conceptual layer.
  - Using a new storage device like Hard Drive or Magnetic Tapes
  - Modifying the file organization technique in the Database
  - Switching to different data structures.
  - Changing the access method.
  - Modifying indexes.
  - Changes to compression techniques or hashing algorithms.
  - Change of Location of Database from say C drive to D Drive

Basic building blocks of DBMS

A data model constitutes of the following building blocks:

1. Entities
2. Attributes
3. Relationships
4. Constraints

1. Entities

- The first building block of a data model is data entities, which represent the objects or concepts that are relevant to the system.
- For example, in an e-commerce system, data entities might include products, customers, orders, and payment transactions.
- Each data entity is composed of attributes, which are characteristics or properties of the entity.
- For example, a product entity might have attributes such as pname, price, and product ID, while a customer entity might have attributes such as cname, email, and phone number.

2. Attributes

- It is the set of characteristics representing an entity.
- *Example:* A student has attributes like name, roll number, age and much more.
- It is important to choose the appropriate data type for each attribute, as it determines how the data can be stored and used.
- For example, if an attribute is intended to store monetary values, it should be defined as a numeric data type, as this will allow mathematical operations to be performed on it.

3. Relationships

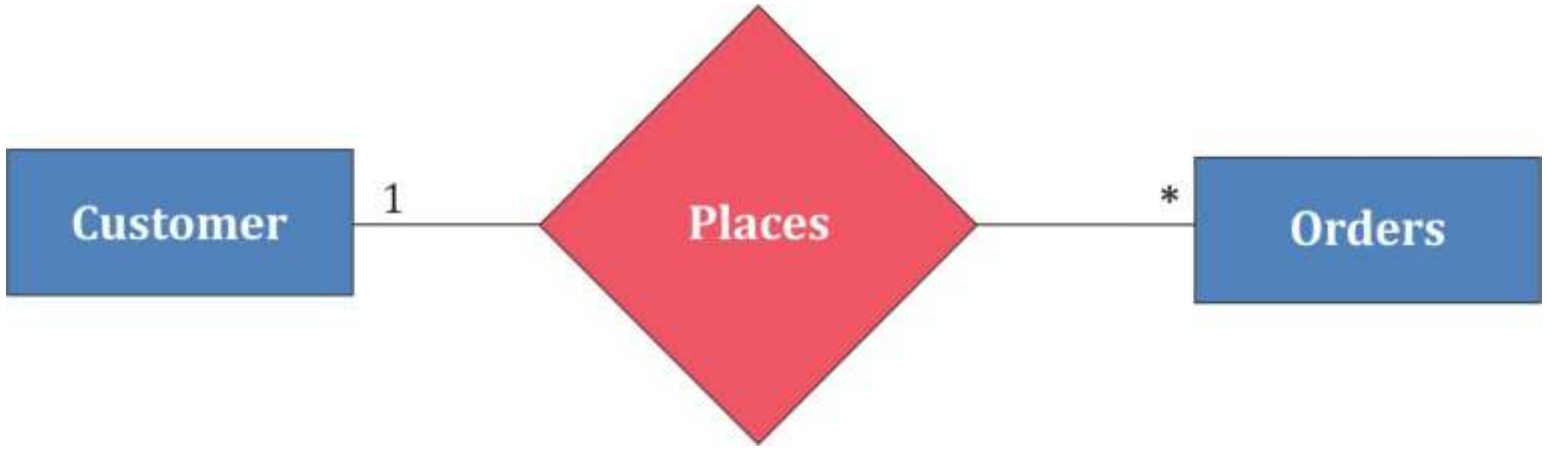
- It describes the association between two entities.
- The data model generally uses three kinds of relationships:
  - one to many
  - many to many
  - one to one.

One to One Relationship (1:1)

- It is used to create a relationship between two tables in which a single row of the first table can only be related to one and only one records of a second table.
- Similarly, the row of a second table can also be related to anyone row of the first table.

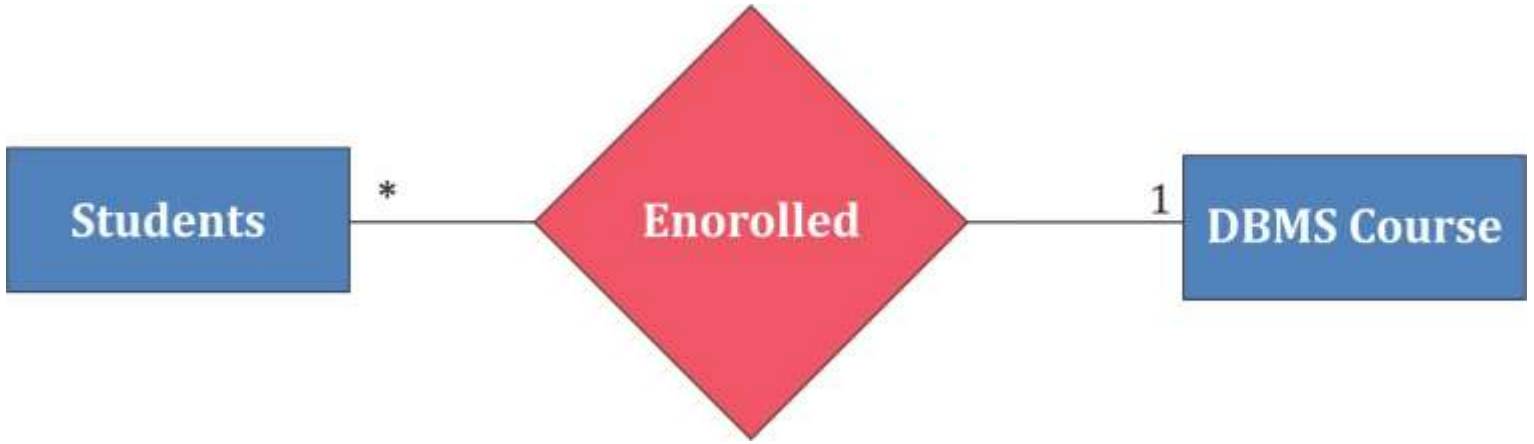
One to Many Relationship (1:M)

- Any single rows of the first table can be related to one or more rows of the second tables, but the rows of second tables can only relate to the only row in the first table.



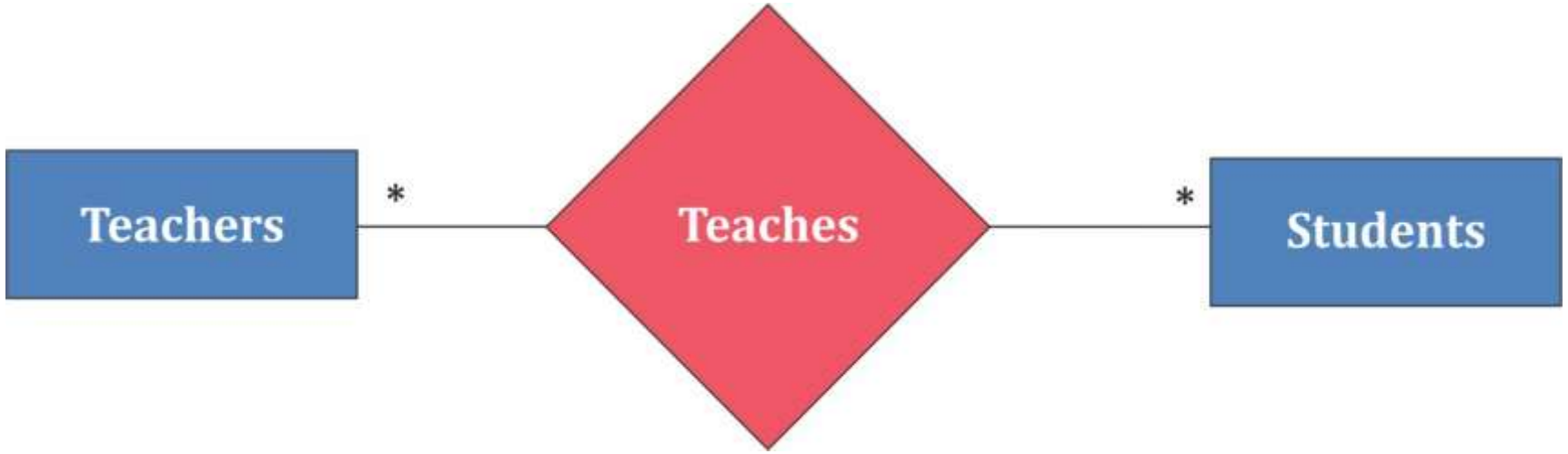
Many to One Relationship (M:1)

- Multiple rows of the first table can be related to any one row of the second tables.



Many to Many Relationship (M:M)

- Each record of the first table can relate to any records (or no records) in the second table.
- Similarly, each record of the second table can also relate to more than one record of the first table.
- It is also represented an **N:N** relationship.



4. Constraints

- Constraints are conditions applied on the data.

- For example, if a product table has a constraint on the "Price" field that specifies that it must be a positive number, it will not be possible to enter a negative price for a product.

There are two types of constraints on the Entity Relationship (ER) model −

- Mapping cardinality or cardinality ratio.
- Participation constraints.

Mapping Cardinality

- It is expressed as the number of entities to which another entity can be associated via a relationship set.
- For the binary relationship set there are entity set A and B then the mapping cardinality can be one of the following −
  - One-to-one
  - One-to-many
  - Many-to-one
  - Many-to-many

Participation Constraints

- Participate constraints are two types as mentioned below −
  - Total participation
  - Partial Participation

Cont…

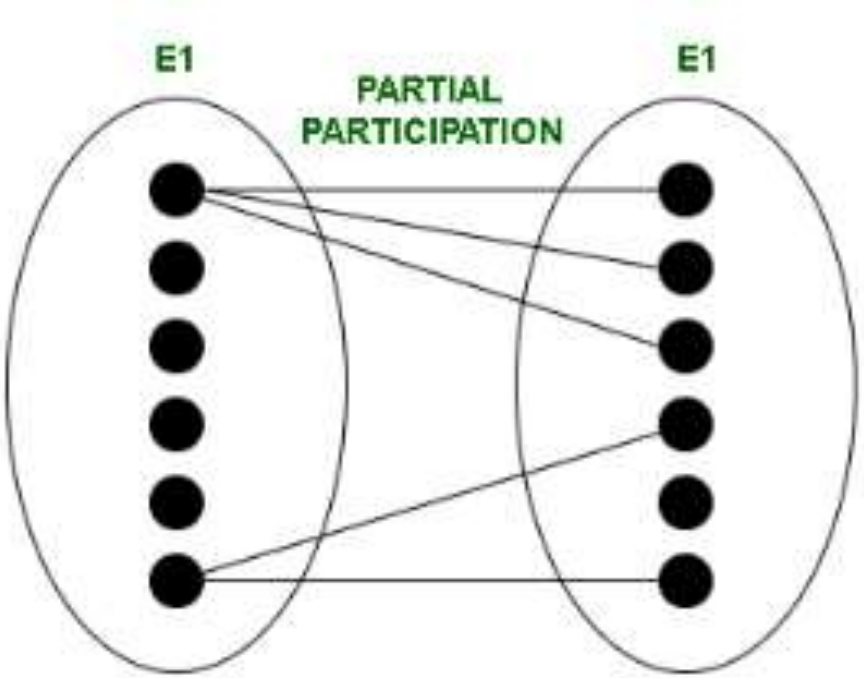
Total participation

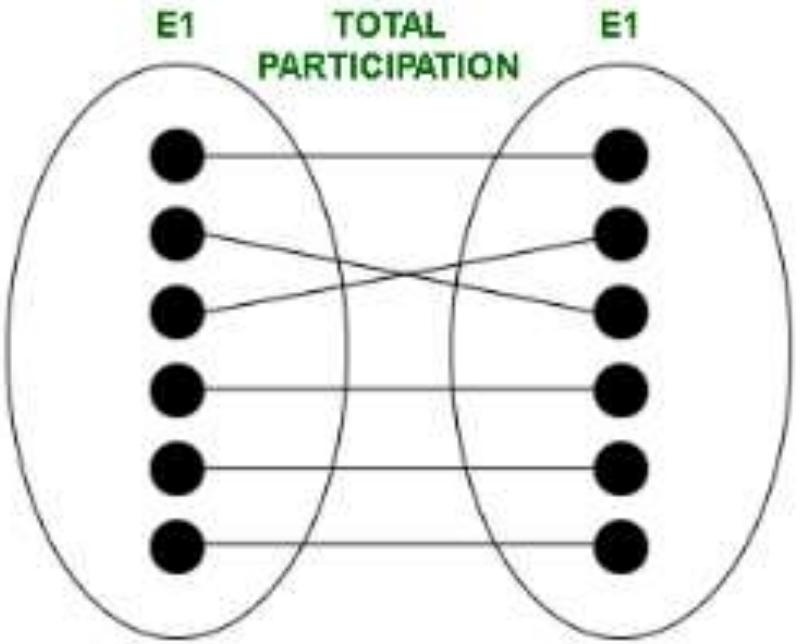
- The participation of an entity set E in a relationship set R is said to be total if every entity in E Participates in at least one relationship in R.
- For Example − Participation of loan in the relationship borrower is total participation.

Partial Participation

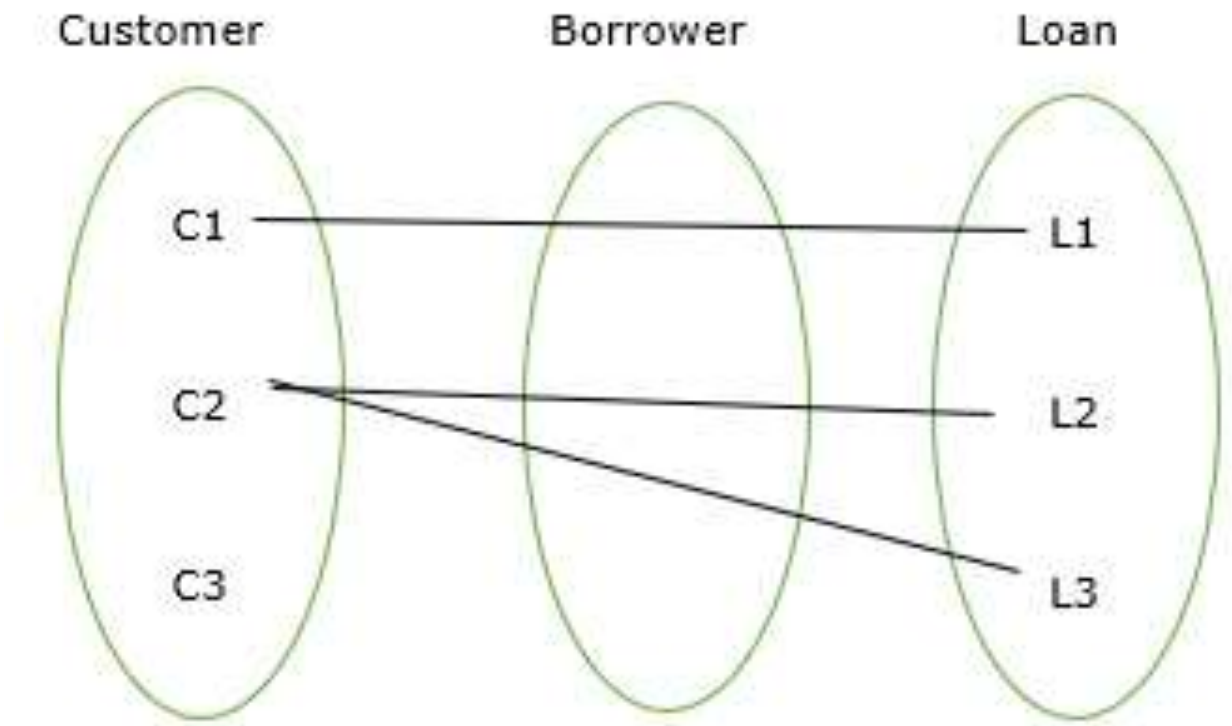
- If only some of the entities in E participate in relationship R, then the participation of E in R is said to be partial participation.
- For example − Participation of customers in the relationship borrower is partial participation.

Cont…





Cont…



ER Model

- **Entity Relationship Model** (ER Modeling) is a graphical approach to database design.
- It is also very easy for the developers to understand the system by just looking at the ER diagram.
- The main components of E-R model are: entity set and relationship set.
- **Entity Relationship Diagram (ERD)** are commonly used in database design to visualize the relationships between entities and their attributes.

Terminology

- An **Entity** is a thing or object in real world that is distinguishable from surrounding environment.
- It can be a person, place, or even a concept.
- ***Example:*** Teachers, Student, Course, Building, Department, etc are some of the entities of a School Management System.
- An **Entity set** is a set of entities of the same type that share the same properties.
  - Example: collection of all students enrolled in the university.
- ***Attributes:*** An entity contains a real-world property called attribute. This is the characteristics of that attribute.
- The entity car has properties like Car_no, Color, Price, etc.

Types of Attributes

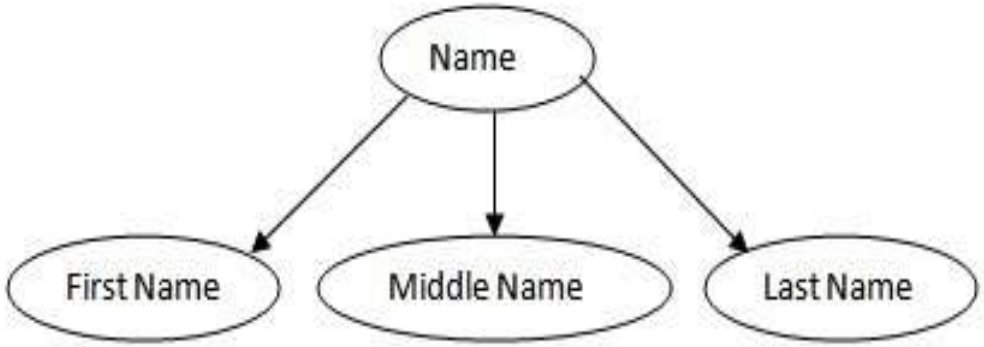
1. Simple Attribute
2. Composite Attribute
3. Single-Valued Attribute
4. Multi-Valued Attribute
5. Derived Attribute
6. Complex Attribute
7. Stored Attribute
8. Key Attribute
9. Null Attribute

Simple Attributes

- An attribute that cannot be further subdivided into components is a simple attribute.
- **Example:** The roll number of a student, the ID number of an employee, gender, and many more.
- These attributes are also known as **atomic attributes.**

Composite Attributes

- Composite attributes have opposite functionality to of simple attributes as we can further subdivide composite attributes into different components or sub-parts that form simple attributes.
- **In simple terms, composite attributes are composed of one or more simple attributes.**

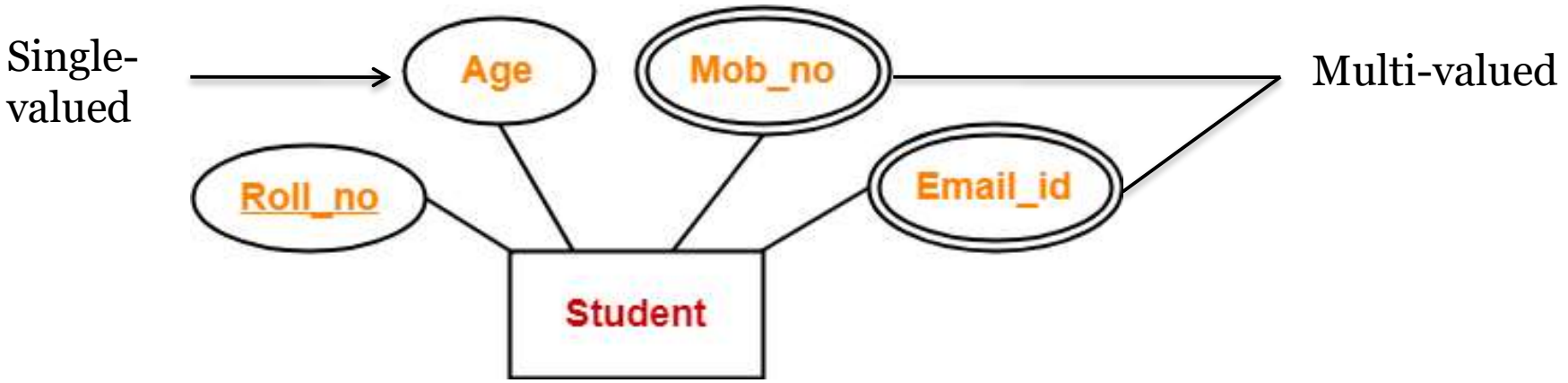


Single-Valued Attributes

- Those attributes which can have exactly one value are known as single valued attributes.
- They contain singular values, so more than one value is not allowed.
- For example, the DOB of a student can be a single valued attribute.
- Another example is gender because one person can have only one gender.

Multi-Valued Attribute

- The attribute which takes up more than a single value for each entity instance is a multi-valued attribute. And it is represented by double oval shape.



Derived Attributes

- Derived attributes in DBMS are those attributes whose values can be derived from the values of other attributes.
- They are always dependent upon other attributes for their value.
- **For example,** From DOB, we can derive the Age attribute, which changes every year, and can easily calculate the age of a person from his/her date of birth value. Hence, the Age attribute here is derived attribute from the DOB single-valued attribute.

Key attribute

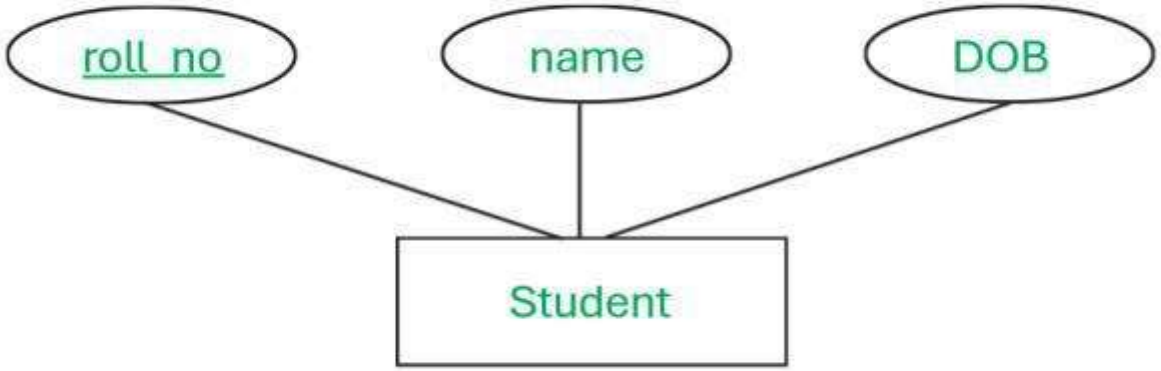
- A key attribute can uniquely identify an entity from an entity set.
- For example, student roll number can uniquely identify a student from a set of students.
- Key attribute is represented by oval same as other attributes however the **text of key attribute is underlined**.

Null Attribute

- The attribute which has no value is known as null attribute.
- For example age of a particular student is not available in the age column of student table then it is represented as null but not as zero
- A null value is used to represent ***the following different interpretations***
  - ***Value unknown*** (value exists but is not known)
  - ***Value not available*** (exists but is purposely hidden)
  - ***Attribute not applicable*** (undefined for that row)

Stored Attribute

- The stored attribute are those attribute which doesn't require any type of further update since they are stored in the database.
- These values help in deriving the derived attributes.



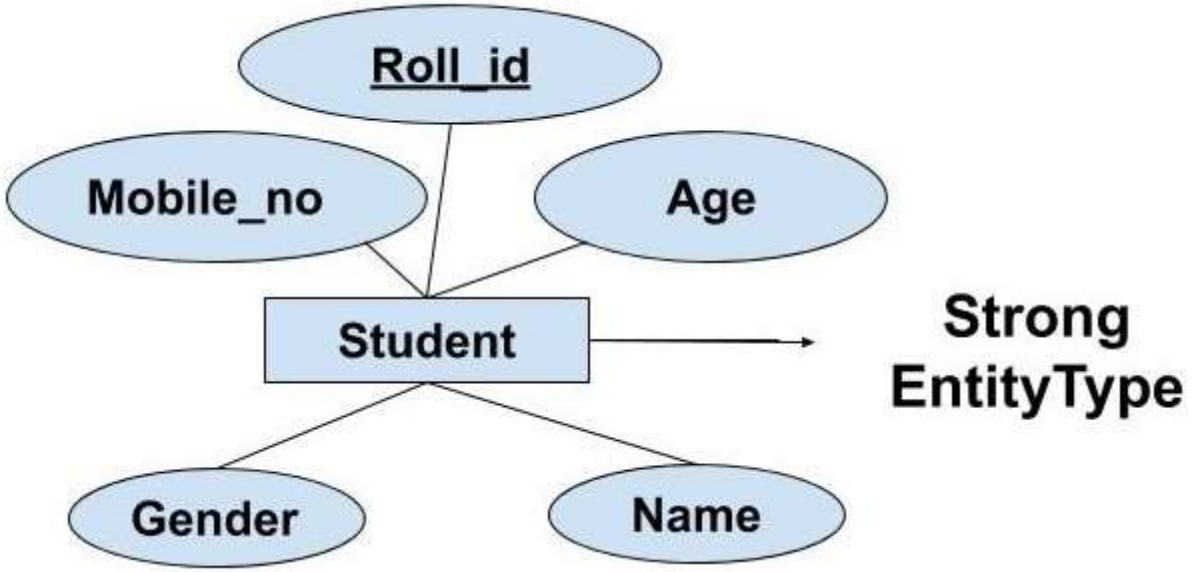
Complex Attribute

- It is formed by nesting composite attributes and multi-valued attributes in arbitrary way. We can say this as the both are in the attribute.
- These attributes are rarely used in DBMS(DataBase Management System).
- **Example**: Address because address contain composite value like street, city, state, PIN code and also multivalued because one people has more that one house address.

Types of Entity

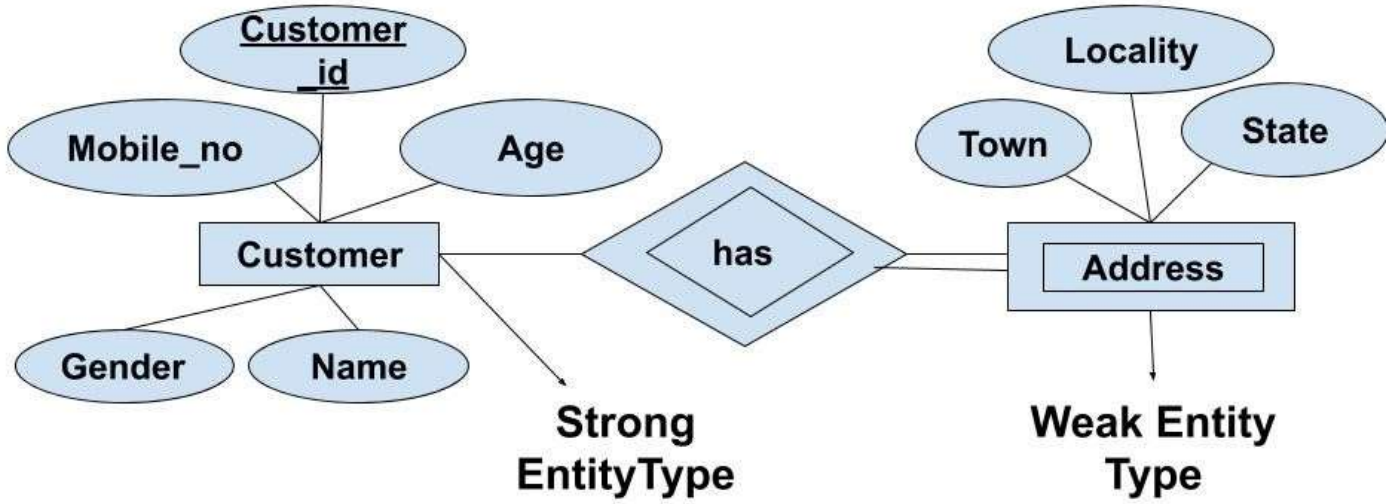
- ***Strong Entity Type:*** Strong entity are those entity types which has a key attribute. The primary key helps in identifying each entity uniquely. It is represented by a rectangle. In the given example, Roll_no identifies each element of the table uniquely and hence, we can say that STUDENT is a strong entity type.
- Strong entity is represented by single rectangle.

Strong Entity

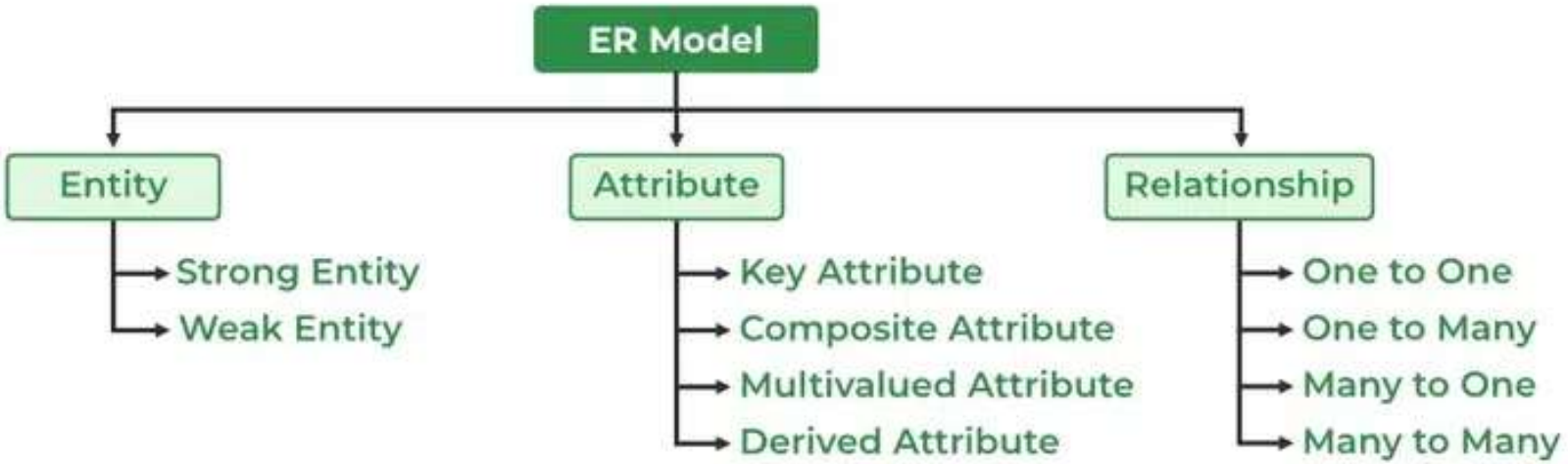


Weak Entity

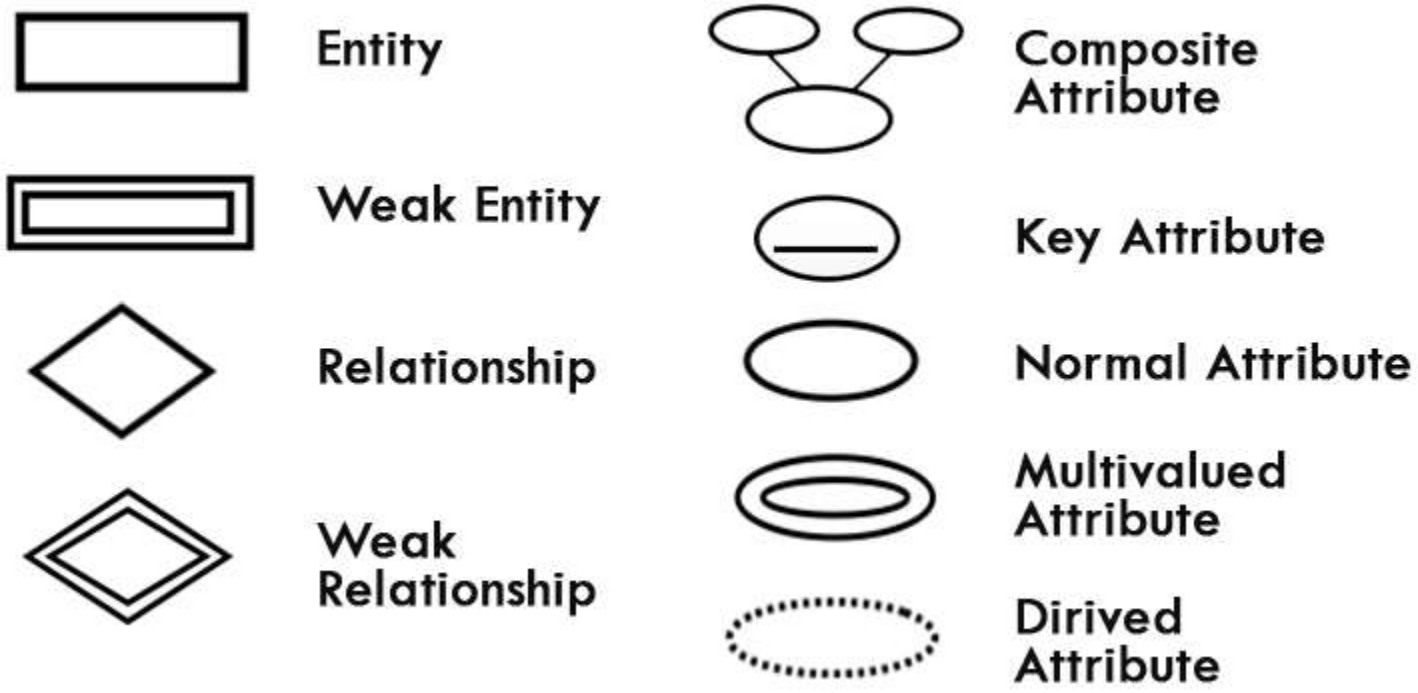
- A weak entity is an entity set that does not have sufficient attributes for Unique Identification of its records.
- Weak entity is represented by double rectangle.



Components of ER Diagram



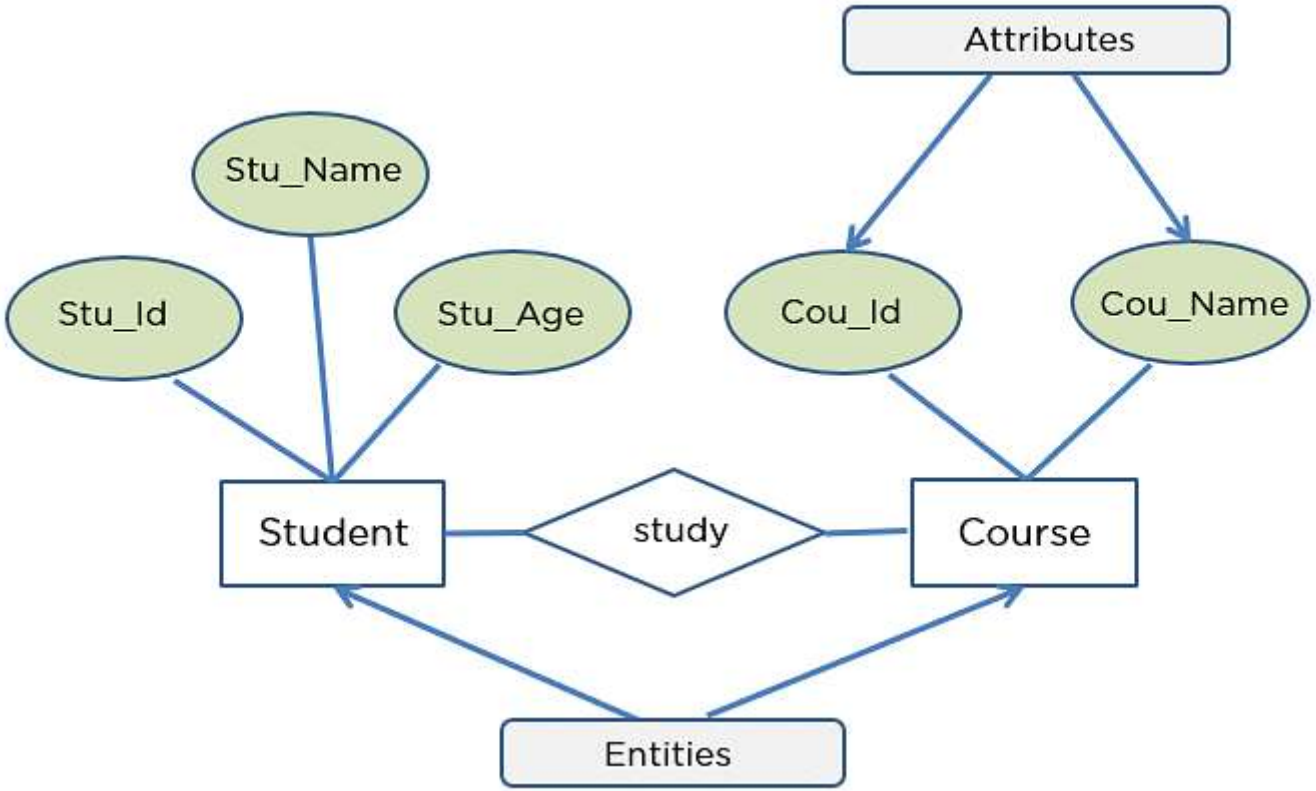
Symbols used in ERD



How to Draw Entity Relationship Diagrams

1. **Step-1**: Determine the Entities in Your ERD
2. **Step-2**: Add Attributes to Each Entity
3. **Step-3**: Define the Relationships Between Entities
4. **Step-4**: Add Cardinality to Every Relationship in your ER Diagram
5. **Step-5**: Finish and Save Your ERD

Example of a Basic ERD



ER Diagram Example-1

The Scenario:

A University contains many Faculties. The Faculties in turn are divided into several Schools. Each School offers numerous programs and each program contains many courses. Lecturers can teach many different courses and even the same course numerous times. Courses can also be taught by many lecturers. A student is enrolled in only one program but a program can contain many students. Students can be enrolled in many courses at the same time and the courses have many students enrolled.

ER Diagram Example-2

Example

- Draw of ER model for company considering the following constraints −
  - In a company, an employee works on many projects which are controlled by one department.
  - One employee supervises many employees.
  - An employee has one or more dependents.
  - One employee manages one department.

Cont…

- Employee – The relevant attributes are name, ssn, gender, address, salary.
- Department – The relevant attributes are Name, number of employees, location.
- Project – The relevant attributes are number, name, location.
- Dependent – The relevant attributes are name, gender, birth date, relationship.

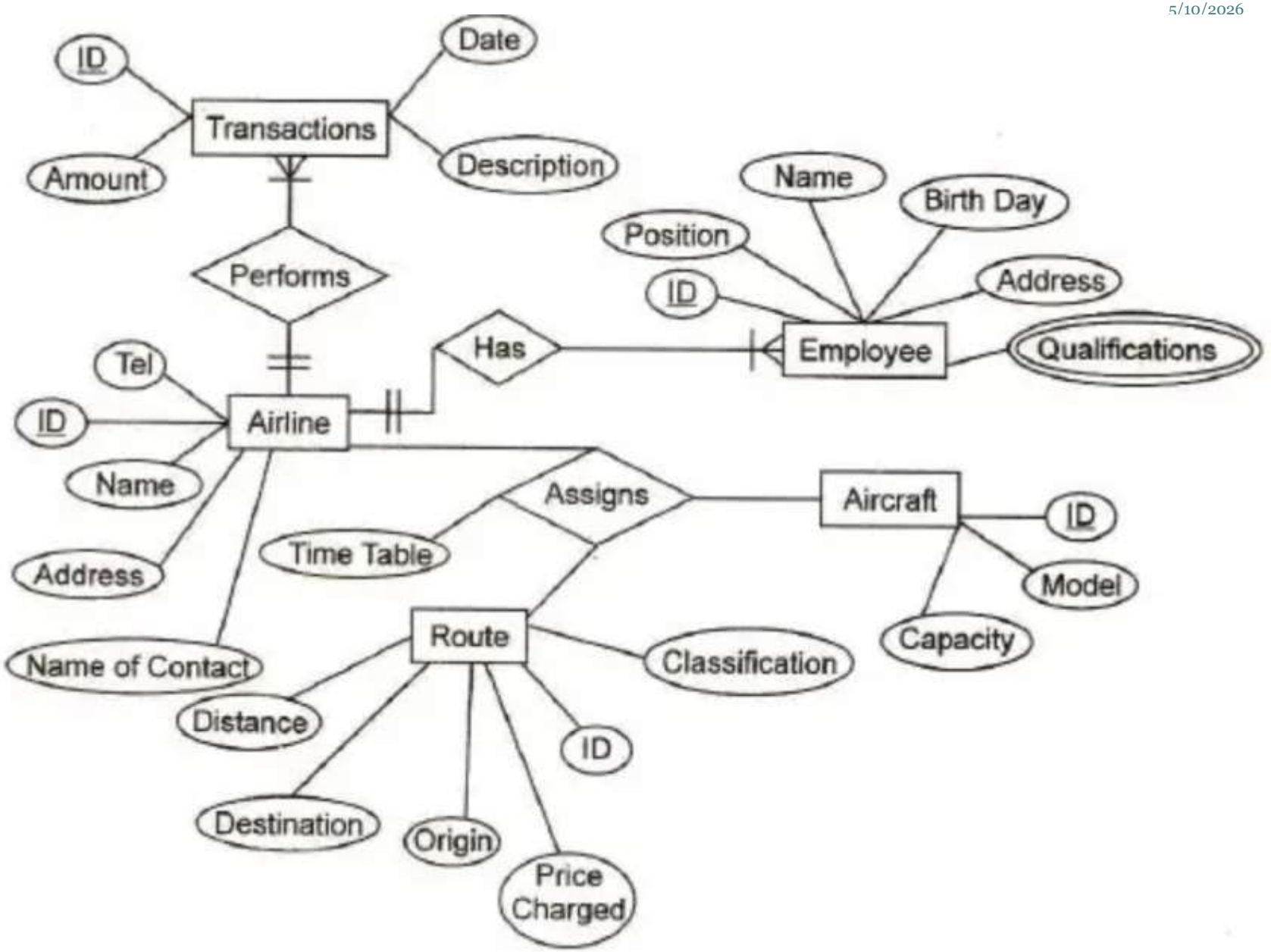
ER Diagram Example-3

Major airlines companies that provide passenger services in Taiwan are: UniAir, TransAsia Airways, Far Eastern Transport, Great China Airlines etc. Taiwan's Federal Aviation Administration (TFAA) keeps a database with lots of information on all airlines. This information is made accessible to all airlines in Taiwan with the intention of helping the Companies assess their Competitive position in the domestic market. The information kept consists of:

1. Each airpline has an identification number, name of the contact person and telephone number.
2. For each aircraft identification number, capacity and model is recorded.

3. Each employee has an employee identification number, name, address, birthday, sex, position with the company and qualification.
4. Each route has a route identification number, origin, destination, classification (into domestic or international route), distance of the route and price charged per passenger.
5. Each airline keeps information about their buy/sell transactions (for example, selling an airplane ticket is a sell transaction, paying for maintenance is a buy transaction). Each transaction has a transaction identification number, date, description and amount of money paid/received.

Draw an E-R diagram for the database presented above. Make Sure to identify the associative entity (entities) and provide corresponding key attribute (attributes)


ID
Date
Transactions
Amount
Description
Performs
Name
Birth Day
Position
Address
ID
Employee
Qualifications
Has
Tel
ID
Airline
Name
Address
Assigns
Aircraft
ID
Time Table
Model
Capacity
Route
Classification
Name of Contact
Distance
ID
Destination
Origin
Price Charged

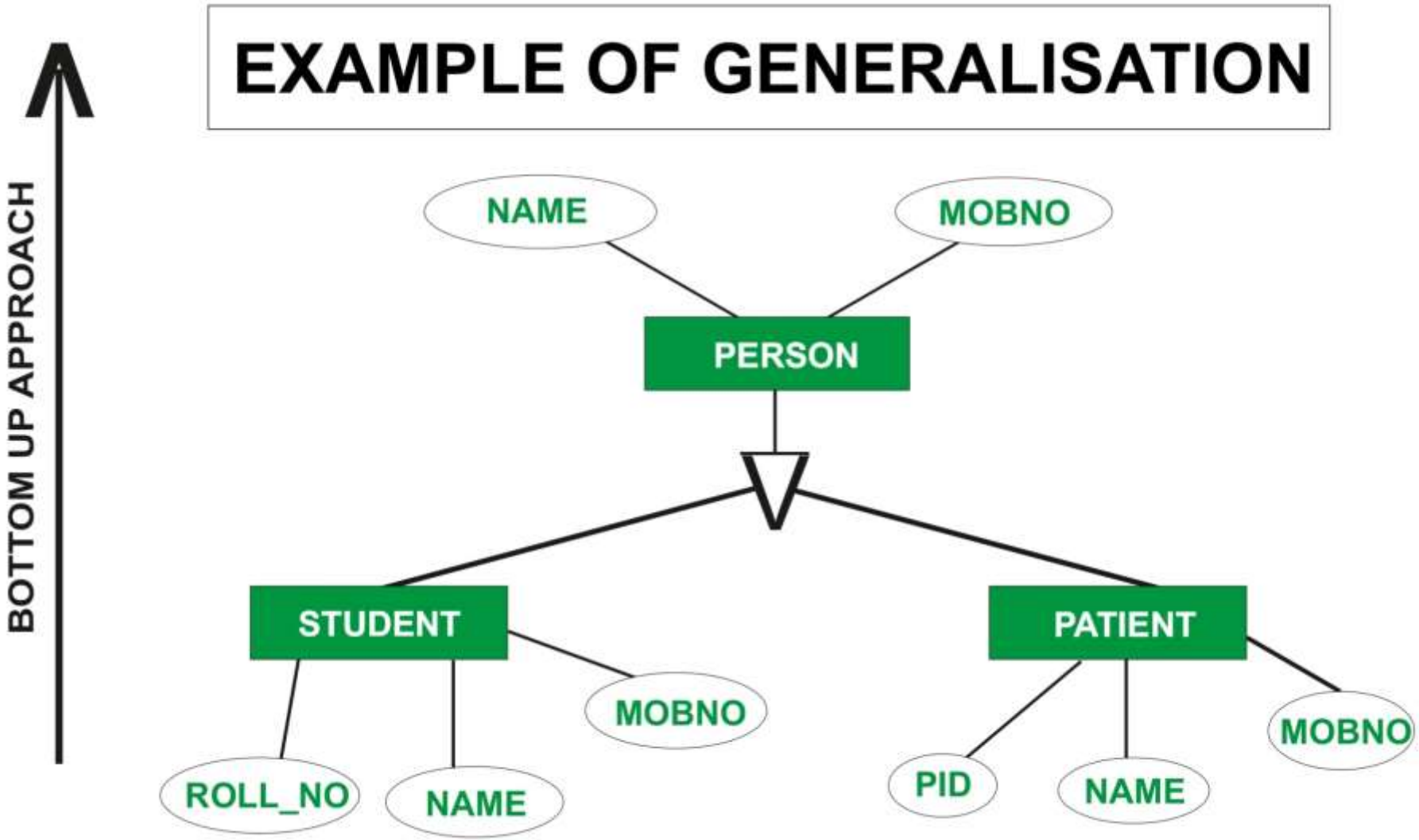
Features of ER model

Generalization:

- Generalization is like a **bottom-up approach** in which two or more entities of lower level combine to form a higher level entity if they have some attributes in common.

Advantage:

- Cut down data redundancy
- Simplified schema


EXAMPLE OF GENERALISATION
BOTTOM UP APPROACH
NAME
MOBNO
PERSON
STUDENT
PATIENT
MOBNO
MOBNO
ROLL_NO
NAME
PID
NAME

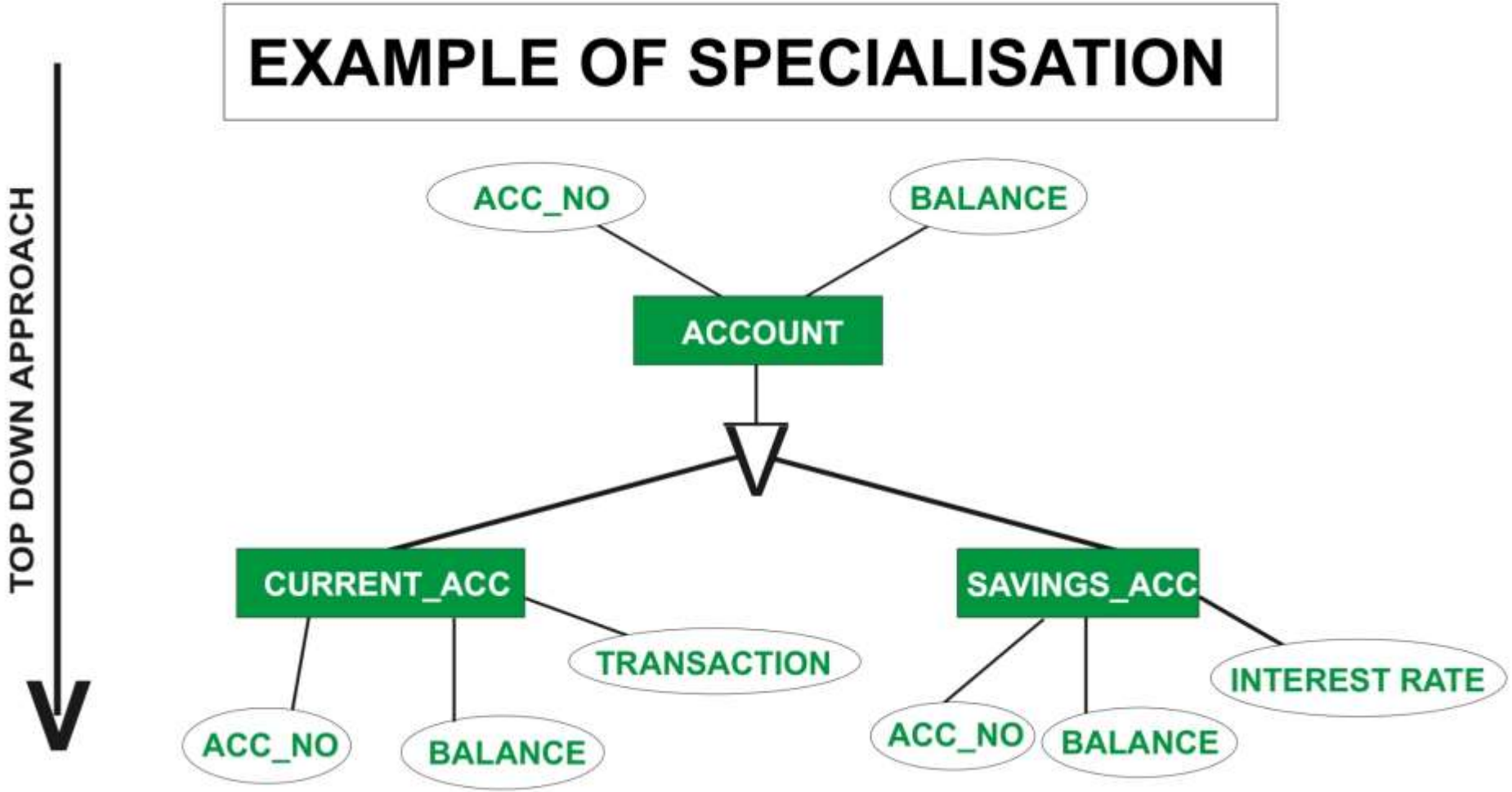
Cont…

Specialization:

- Specialization is a **top-down approach**, and it is opposite to Generalization.
- In specialization, one higher level entity can be broken down into two lower level entities.

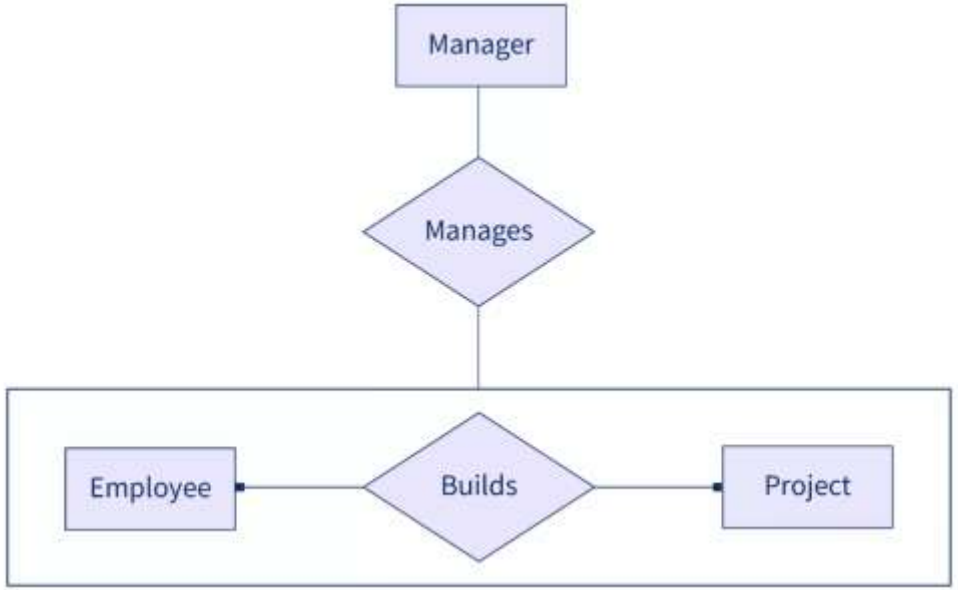
Advantage:

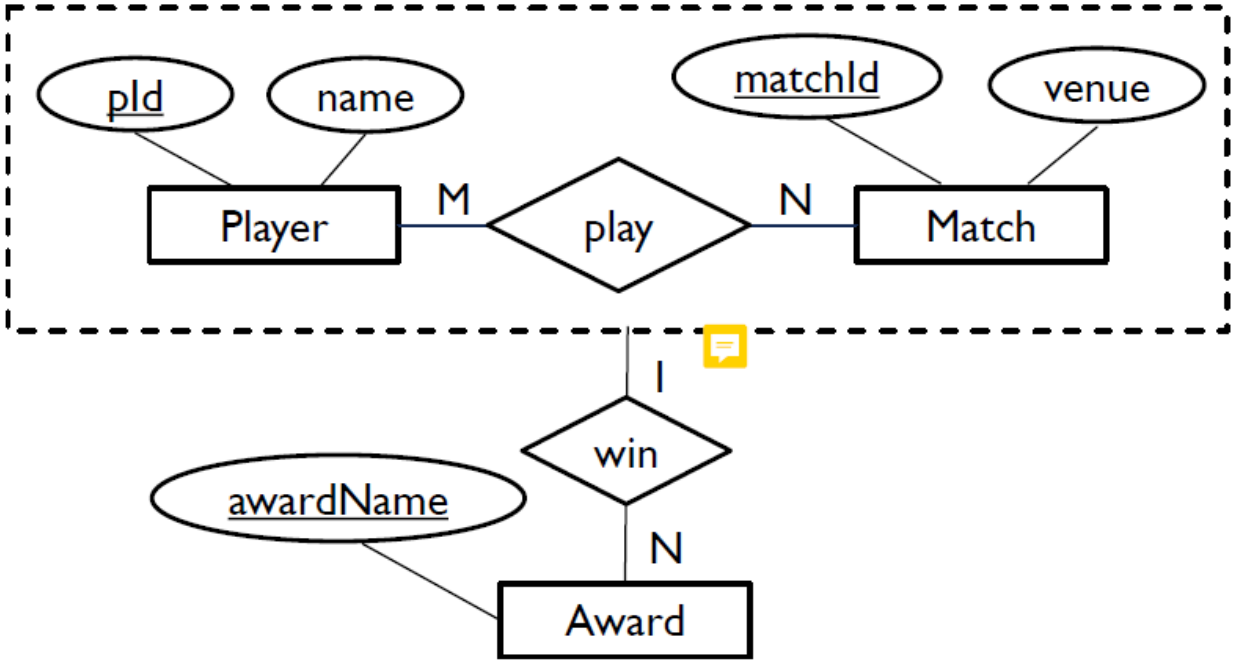
- Enhances Specificity
- Encourages Inheritance


EXAMPLE OF SPECIALISATION
TOP DOWN APPROACH
ACC_NO
BALANCE
ACCOUNT
CURRENT_ACC
SAVINGS_ACC
TRANSACTION
INTEREST RATE
ACC_NO
BALANCE
ACC_NO
BALANCE

Aggregation

- In aggregation, the relation between two entities is treated as a single entity.
- In aggregation, relationship with its corresponding entities is aggregated into a higher level entity.


Manager
Manages
Employee
Builds
Project


pId
name
matchId
venue
Player
M
play
N
Match
1
win
awardName
N
Award

What is Key in RDBMS?

- Keys are one of the basic requirements of a relational database model.
- It is widely used to identify the tuples(rows) uniquely in the table.
- We also use keys to set up relations amongst various columns and tables of a relational database.

Relational Keys

- Candidate Key
- Primary Key
- Alternative Key
- Super Key
- Foreign Key

Candidate Key

- **CANDIDATE KEY** is a set of all the unique attributes in a table.
- It must contain unique values but it can be NULL.
- The Primary key should be selected from the candidate keys. Every table must have at least a single candidate key.
- A table can have multiple candidate keys but only a single primary key.

Example

- In the given table **Stud ID, Roll No, and email** are candidate keys which help us to uniquely identify the student record in the table.

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

Primary key

- **PRIMARY KEY** is a column that uniquely identifies every row in that table.
- The primary key field is always **Unique and Not NULL**.
- The Primary Key can't be a duplicate meaning the same value can't appear more than once in the table.
- **A table cannot have more than one primary key.**

Alternate Key

- **ALTERNATE KEYS** is a column or group of columns in a table that uniquely identify every row in that table.
- All the keys which are not primary key are called an Alternate Key.

Example:

- In this table, StudID, Roll No, Email are qualified to become a primary key. But since StudID is the primary key, Roll No, Email becomes the alternative key.

Super Key

- **Super Key** is defined as a set of attributes within a table that can uniquely identify each record within a table.
- Super Key is a superset of Candidate key.
- In the table defined above super key would include student_id, name, email etc.
- (student_id, name): name of two students can be same, but their student_id can't be same hence this combination can also be a key which is super key.
- Similarly (student_id, email, name), (email, name) can also be super keys.

Foreign Key

- **Foreign key** is the column of the table which is used to refer to the primary key of another table.

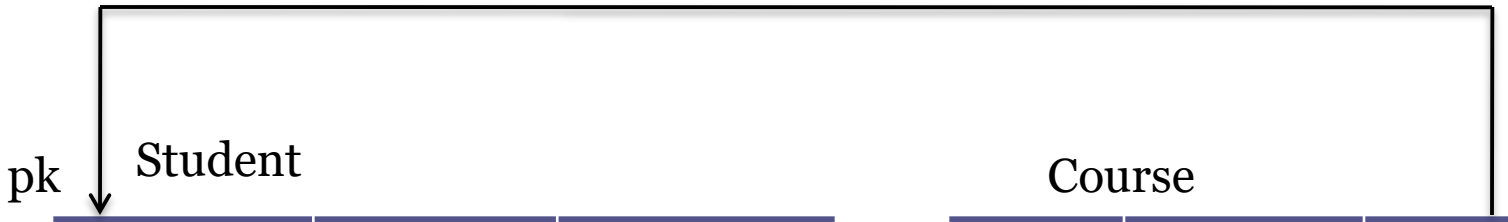
Example

- In Student entity, every student's Regd_no, Name, Address is there. Course is another entity. So we can't store the course-related information in the student table. That's why we link these two tables through the primary key of one table.
- So we can add a new attribute, i.e. Regd_no in the Course table which takes reference from the primary key of Student table.
- So here, Regd_no becomes the foreign key.

Cont…

- Foreign key helps to maintain the **Referential integrity.**
- Every relationship in the model needs to be supported by a foreign key.
- A table can **have more than one** foreign keys.

Concept of Referential Integrity



Regd_no	Name	Address
1	A	Delhi
2	B	Bangalore
3	B	Hyderabad

(Referenced Table)

Cid	Cname	Regd_no
c1	dbms	1
c2	C++	2
c3	python	3

(Referencing Table)

Referenced Table	Referencing Table
1. Insertion: No violation	1. Insertion: May cause problem
2. Deletion: May cause problem	2. Deletion: No violation
3. Updation: May cause problem	3. Updation: May cause problem

Overview of File Structure in Database

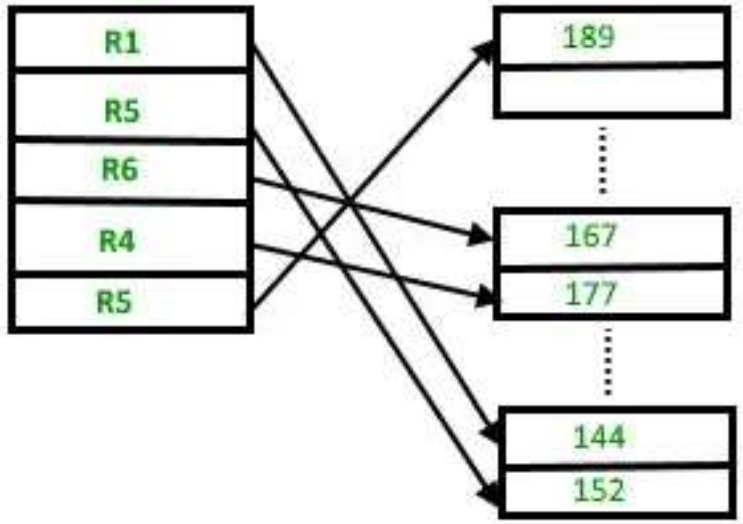
- In **DBMS (Database Management System)**, the **file structure** refers to how **data is stored on disk** in the form of **files**, and how the DBMS **accesses and organizes** that data efficiently.

- A DBMS handles **large volumes of data**, so the way it **stores, retrieves, updates, and organizes** data in files directly impacts:
  - Performance
  - Storage efficiency
  - Query processing speed

Types of File Structures in DBMS

1. Heap File (Unordered File)

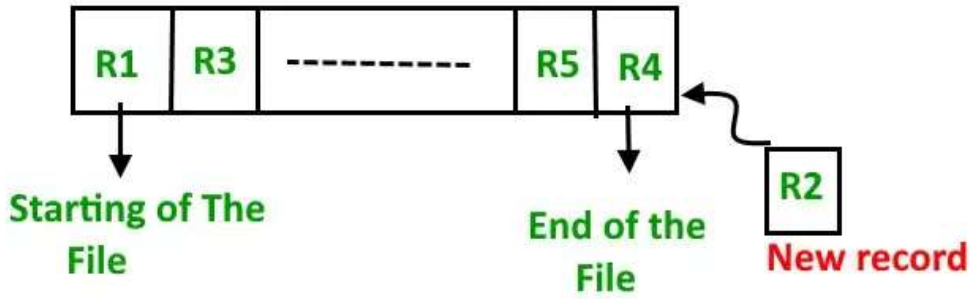
- works with data blocks.
- In this method, records are inserted at the end of the file, into the data blocks.
- No Sorting or Ordering is required in this method. If a data block is full, the new record is stored in some other block



Cont…

2. Sequential File

- In this method, the file is stored one after another in a sequential manner.
- It can be sorted or unsorted.



Pile method

Sorted method

Cont…

3. Hashed File

- Data is stored at the data blocks whose address is generated by using **a hash function**.
- The memory location where these records are stored is called a **data block or data bucket.**

Types of Hashing:

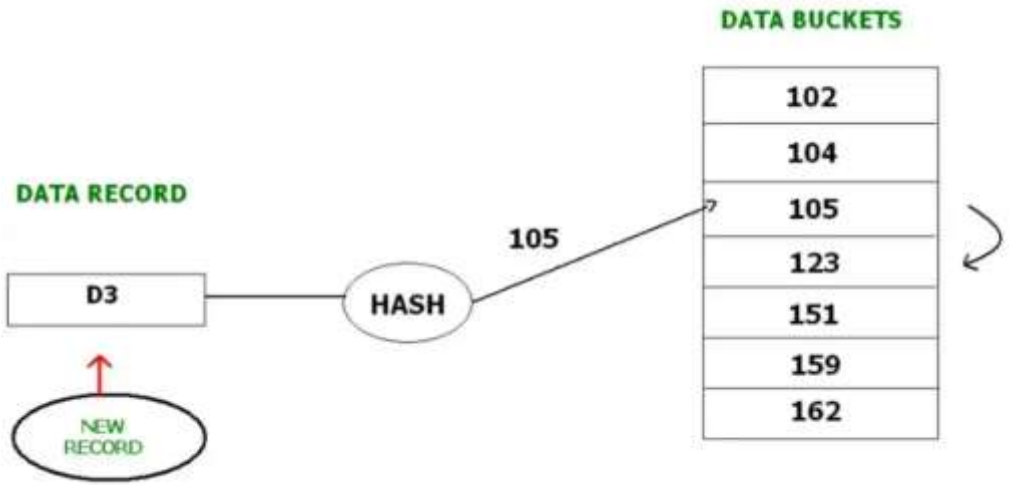
A. Static Hashing

- Number of buckets is fixed
- Simple but not good if file grows
- More chances of overflow

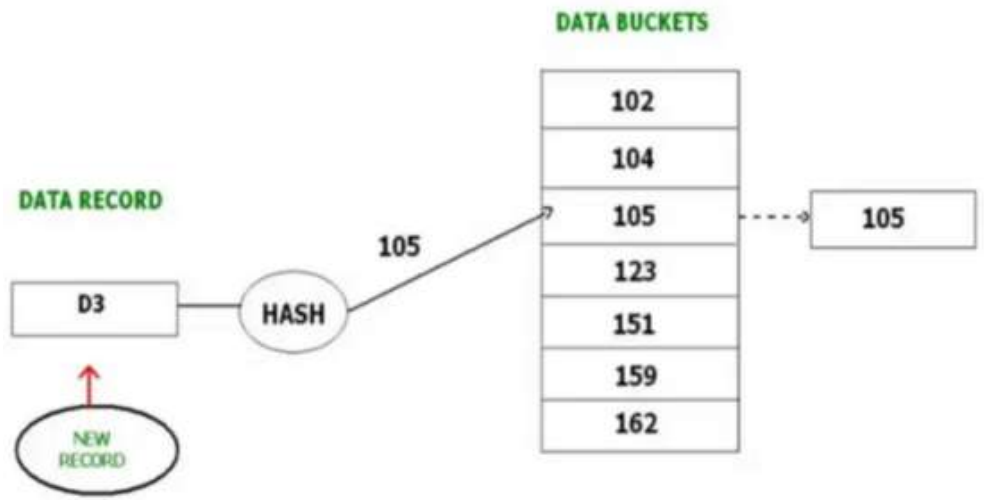
B. Dynamic Hashing

- Number of buckets can grow or shrink
- Avoids overflow problem
- Used in modern databases

Techniques to handle collision:



Closed hashing/ Open Addressing

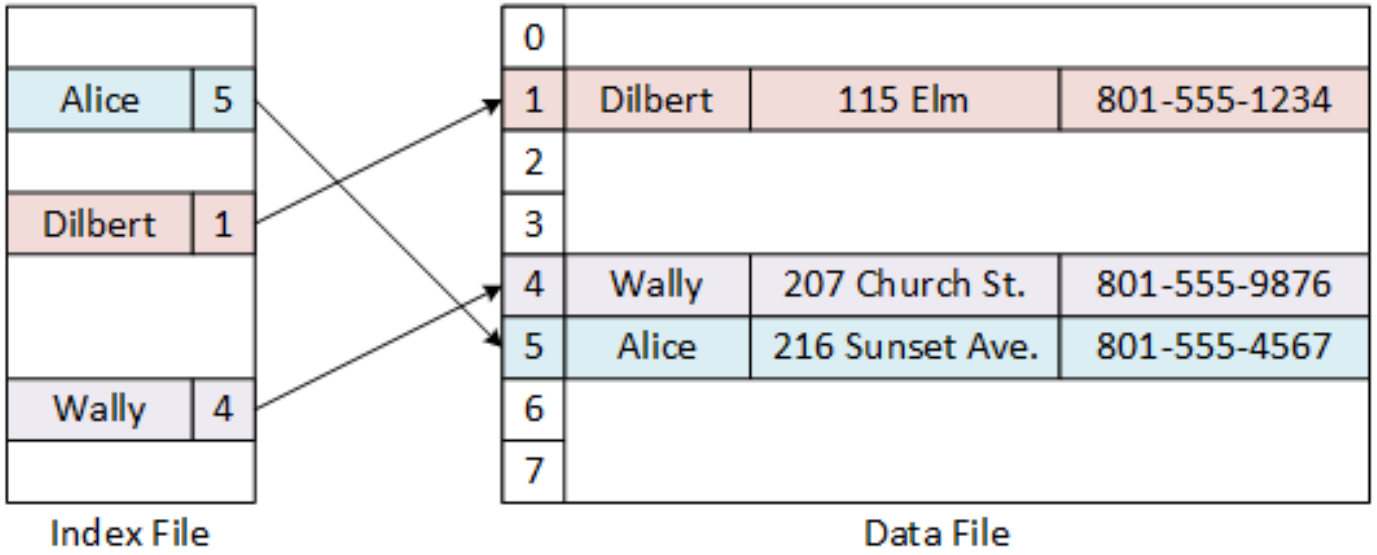


Open hashing/ Chaining

Cont…

4. Indexed File

- Adds **indexes** to improve search performance.
- An index file stores **pointers to records** in the main data file.
- Can be used with heap, sequential, or hashed files.



Types of Indexing

1. **Primary Indexing:** This is a type of indexing wherein the data is sorted according to the search key and the primary key of the database table is used to create the index.
2. **Clustered Indexing:** Data is stored in sorted order based on often a non-primary key.
3. **Secondary Indexing:** Secondary index may be generated from a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values. It can be sorted or unsorted.

End of Unit-1